TÉCNICO LISBOA



TANTGen: Transformer-based Adversarial Network Traffic Generator

Inês de Almeida Pissarra

Thesis to obtain the Master of Science Degree in

Computer Science and Engineering

Supervisor: Prof. Fernando Manuel Valente Ramos

Examination Committee

Chairperson: Prof. Name of the Chairperson
Supervisor: Prof. Fernando Manuel Valente Ramos
Member of the Committee: Prof. Name of First Committee Member

April 2026

Declaration
I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

Acknowledgments

I would like to express my heartfelt gratitude to all those who supported me throughout the development of this thesis.

First and foremost, I am deeply grateful to my thesis advisor, Prof. Fernando Ramos, whose guidance, expertise, and constructive feedback were instrumental in this work.

A special thanks to my mother, who, despite not being from the academic field, always listened attentively and offered help and encouragement whenever needed. I am also very grateful to my brother for his genuine concern and involvement in my research, and to my father for his sincere interest and fascination with everything I was working on.

I want to thank my friend Ricardo Pereira for his constant moral support and for lightening challenging moments with his thoughtful gestures.

I am sincerely grateful to my university colleagues and friends Ana, Madalena, and Juliana for their insightful discussions and for sharing both the challenges and successes of this journey with me.

I would also like to express my gratitude to my colleague and friend Guilherme Leitão for his support and the trust he placed in my work. His decision to develop defenses based on the attack model I created — even before my own work was completed — was a motivating factor in finalizing this thesis.

Finally, to all my family and friends, thank you for your unwavering support.

Without the contribution of each of you, this thesis would not have been possible.

Abstract

Machine Learning (ML) has made notable advances, with applications across a broad range of fields, including cybersecurity. As new attacks emerge, several ML-based techniques have been studied to detect known and zero-day attacks. Although ML has shown excellent results, it has also demonstrated vulnerabilities to adversarial attacks, where inputs are slightly perturbed to induce misclassification. New research has explored adversarial attack strategies to identify these vulnerabilities and improve model robustness. However, many existing approaches rely on impractical assumptions, such as attacker access to model parameters, or fail to ensure that attacker-induced changes comply with network protocol rules and semantics.

In this thesis, we present TANTGen, a framework to generate realistic adversarial network traffic targeting ML-based Network Intrusion Detection Systems. A key innovation is the Transformer architecture, which excels at modeling complex sequences with intricate dependencies, enabling the capture of relationships between network features to generate effective attacks. TANTGen operates under a black-box threat model, where the attacker has no knowledge of the target model, while preserving protocol semantics. TANTGen achieves attack success rates of up to 99.9% when no perturbation constraints are imposed, and outperforms TrafficManipulator [1] under stricter conditions, achieving an Attack Success Rate of 99.9% against 75.2% under a $6\times$ of the original time constraint and 22.2% against 18.1% under a $3\times$ constraint. By enabling the generation of realistic adversarial attacks, TANTGen provides a foundation for systematically studying model vulnerabilities and improving the robustness of ML-based detection systems.

Keywords

Machine Learning; Adversarial Attack; Intrusion Detection System; Transformer

Resumo

A aprendizagem automática (ML) tem registado avanços notáveis em diversas áreas, incluindo cibersegurança. À medida que surgem novos ataques, várias técnicas baseadas em ML têm sido estudadas para detetar ataques conhecidos e de dia zero. Embora a ML tenha demonstrado excelentes resultados, também revelou vulnerabilidades a ataques adversariais, nos quais as características de entrada são ligeiramente perturbadas de forma a que o modelo as classifique incorretamente. No entanto, muitas abordagens existentes assentam em pressupostos pouco realistas, como o acesso do atacante aos parâmetros do modelo, ou não garantem que as alterações induzidas respeitem as regras e a semântica dos protocolos de rede.

Nesta tese, apresentamos o TANTGen, um framework para gerar tráfego de rede adversarial realista para atacar sistemas de deteção de intrusão (NIDSs) baseados em ML. A inovação principal é a utilização da arquitetura Transformer, que se destaca na modelação de sequências complexas, permitindo capturar relações entre características de rede para gerar ataques eficazes. O TANTGen opera em caixa negra, sem conhecimento do modelo alvo, preservando a semântica dos protocolos de rede. Atinge taxas de sucesso de até 99,9% sem restrições de perturbação e supera o TrafficManipulator [1] em condições mais restritivas, alcançando um ASR de 99,9% contra 75,2% sob uma restrição de $6\times$ do tempo original e 22,2% contra 18,1% sob $3\times$. Ao permitir a geração de ataques adversariais realistas, o TANTGen fornece uma base para o estudo sistemático das vulnerabilidades dos modelos e para a melhoria da robustez dos sistemas de deteção baseados em ML.

Palavras Chave

Aprendizagem Automática; Ataque Adversarial; Sistema de Deteção de Intrusão; Transformer

Contents

1 Introduction **1**

1.1 Work Objectives and Challenges . . . 2

1.2 Rationale and Contributions . . . 3

1.3 Organization of the Document . . . 4

2 Background **5**

2.1 Machine Learning (ML) . . . 5

2.1.1 Data Preparation and Representation . . . 6

2.1.2 Model Training and Deployment . . . 7

2.2 Adversarial Attacks on Machine Learning . . . 10

2.2.1 Types of Adversarial Attacks . . . 11

2.2.2 Defenses against Adversarial Attacks . . . 12

2.3 ML-based networks . . . 12

2.4 Summary . . . 15

3 Related Work **17**

3.1 Adversarial Attacks in ML-based Networks . . . 17

3.1.1 White-box Feature-space attack . . . 18

3.1.2 White-box Traffic-space attack . . . 19

3.1.3 Black-box Feature-space attack . . . 20

3.1.4 Black-box Traffic-space attack . . . 21

3.2 Use of Transformers for Networking Tasks . . . 23

3.3 Summary . . . 24

4 Solution **27**

4.1 Threat model . . . 27

4.2 TANTGen Architecture Overview . . . 28

4.2.1 Data Representation and Problem Formulation . . . 29

4.2.2 Model selection . . . 31

4.2.3 Transformer Training . . . 32

4.2.4 Attack application . . . 34
4.3 Implementation . . . 35
4.3.1 Data Pipeline and Traffic Reconstruction . . . 35
4.3.2 Transformer-Based Adversarial Model . . . 35
4.4 Summary . . . 36
5 Evaluation **37**
5.1 Experimental Settings . . . 37
5.1.1 Target Network Intrusion Detection Systems (NIDSs) and Feature Extractors . . . 38
5.1.2 Datasets . . . 38
5.1.3 Attacker's Capability . . . 40
5.1.4 Baseline . . . 41
5.1.5 Setup . . . 42
5.1.6 Research Questions and Metrics . . . 43
5.2 Results . . . 44
5.2.1 Adversarial Results on CIC-IDS2017 . . . 44
A – Impact of size perturbation. . . . 46
B – Impact of IAT perturbation. . . . 47
C – Impact of combined perturbation. . . . 47
D – Effect of size perturbation constraints. . . . 47
5.2.2 Adversarial Results on Kitsune . . . 49
A – Impact of size perturbation. . . . 51
B – Impact of IAT perturbation. . . . 51
C – Impact of combined perturbation. . . . 52
D – Comparison with TrafficManipulator. . . . 53
5.2.3 Realism and Maliciousness . . . 54
A – Realism. . . . 54
B – Maliciousness. . . . 54
5.2.4 Operational Feasibility . . . 55
A – Time breakdown. . . . 55
B – Throughput and latency. . . . 55
C – Resource usage. . . . 57
5.3 Summary . . . 57
6 Conclusion **59**
6.1 Main Findings . . . 59
6.2 System Limitations and Future Work . . . 60

6.2.1 Maliciousness Verification 61
6.2.2 Feature Space Expansion 61
6.2.3 Adversarial Robustness of NIDS 61
6.2.4 Real-Time Deployment 61
6.2.5 Reinforcement Learning 62

Bibliography **63**

List of Figures

2.1 Machine Learning pipeline 6
2.2 Three basic machine learning paradigms 8
2.3 Machine Learning Models 10

3.1 Attacker types 18

4.1 TANTGen Overview 29
4.2 Data Representation Overview 31
4.3 TANTGen training process 32

5.1 TANTGen Evaluation 42
5.2 Baseline Detection Performance on CIC-IDS2017 45
5.3 Attack Success Rate on CIC-IDS dataset 46
5.4 Difference of Attack Success Rate if force IAT perturbation (only IAT was perturbed here) 48
5.5 Difference of Attack Success Rate if limit size perturbation (only size was perturbed here) 49
5.6 Baseline Detection Performance on Kitsune dataset 50
5.7 Attack Success Rate on Kitsune dataset 51
5.8 Difference of Attack Success Rate if limit IAT perturbation (perturbation only in size) . . . 52
5.9 TANTGen vs TrafficManipulator — Mirai Botnet (Kitsune) 53
5.10 Relative contribution of each processing stage to total execution time 56
5.11 Throughput across attack types and perturbation strategies 56

List of Tables

3.1 Comparison between existing research 25

5.1 TANTGen Hyperparameters 43
5.2 Attack Success Rate results on CIC-IDS Dataset 45
5.3 Mirai Botnet's Attack Success Rate (ASR) results against Kitsune 50
5.4 SSDP Flood's ASR results against Kitsune 50
5.5 End-to-End Computational Cost of TANTGen Across Attack Types and Perturbation Strategies 55

Acronyms

AML	Adversarial Machine Learning
ASR	Attack Success Rate
CNN	Convolutional Neural Networks
DDoS	Distributed Denial-of-Service
DoS	Denial-of-Service
DT	Decision Tree
EDA	Exploratory Data Analysis
GAN	Generative Adversarial Network
IAT	Inter-Arrival Time
IoT	Internet of Things
LR	Logistic Regression
ML	Machine Learning
MLP	Machine Learning Perceptron
MTU	Maximum Transmission Unit
NIDS	Network Intrusion Detection System
NLP	Natural Language Processing
NTT	Network Traffic Transformer
PCA	Principal Component Analysis
PGD	Projected Gradient Descent
PSO	Particle Swarm Optimization
QoE	Quality of Experience
QoS	Quality of Service
RF	Random Forest

RTT	Round-Trip Time
RTTF	Remaining Time To Failure
SSDP	Simple Service Discovery Protocol
SVM	Support Vector Machine
VAE	Variational Autoencoder
WGAN	Wasserstein Generative Adversarial Network
ZOO	Zeroth-Order Optimization

1

Introduction

Contents

1.1 Work Objectives and Challenges 2

1.2 Rationale and Contributions 3

1.3 Organization of the Document 4

Nowadays, Machine Learning (ML) is widely applied across a broad range of domains, from medical diagnostics to the creation of Autonomous Vehicles. With the creation of increasingly flexible and general-purpose models, its use in the computer networking field has also increased.

Network Intrusion Detection Systems (NIDSs) are an example of systems that leverage Machine Learning to detect attacks in network environments, but other applications in computer networking also exist, including traffic engineering, routing, resource and fault management, among others [2]. However, as the use of ML continues to expand, researchers have also explored methods to deceive these ML-powered systems by subtly altering their input features in ways that are imperceptible to the monitoring systems (or to the human eye) but can significantly disrupt system performance. These adversarial attacks have demonstrated critical vulnerabilities in the application of Machine Learning across many domains. Exploring these vulnerabilities is relevant, as it enables the evaluation and improvements of defensive mechanisms, ultimately enhancing their robustness and reliability against real-world threats.

Adversarial attacks have been widely studied in domains such as computer vision [3–5], where attackers manipulate image pixel values to deceive classifiers. In this context, generating adversarial examples is relatively straightforward because the features used by the model are the pixel values themselves, which are differentiable and can be easily adjusted without violating any constraints. These pixel-level perturbations remain imperceptible to humans while effectively compromising the model's performance.

1.1 Work Objectives and Challenges

The goal of this dissertation is to generate adversarial traffic designed to evade ML-based networks. However, extending existing techniques to network traffic [1, 6–14] presents several challenges. Since network traffic is often encrypted, the features extracted to feed ML models are typically based on characteristics such as packet headers, packet sizes, interarrival patterns, and other forms of network metadata. Unlike image pixels, these features are derived rather than directly observed, which introduces additional complexity. This leads to the**inverse feature mapping problem**, where adversarial modifications applied in the feature space must be accurately translated back into realizable network traffic while preserving protocol compliance and semantic correctness.

Some studies [9, 10, 13] bypass this challenge by applying modifications directly to the extracted features and feeding them to the target (in most cases, a component of a NIDS). This scenario is unrealistic, as it assumes a strong adversary with access to the internal feature extraction and model pipeline. In contrast, more recent work [1,6–8,11,12,14] explicitly considers the inverse feature mapping problem and applies modifications directly to network traffic. While more realistic, these approaches still exhibit important limitations, particularly in terms of scalability, efficiency, or the ability to preserve traffic semantics.

Another unrealistic assumption made in several studies [6, 8, 10, 13, 14] is that the adversary has full knowledge of the model and its parameters, allowing the generation of attacks that are specifically tailored to the target model. This type of approach is referred to as a white-box attack. On the one hand, from a defensive perspective, the model must prove to be robust even in the worst-case scenario. On the other hand, black-box attacks, where the adversary has no prior knowledge of the model being used [1, 7, 9–12], are generally considered more realistic and, due to their ability to transfer across models, may pose a greater practical threat when effective. Some studies adopt an intermediate setting, referred to as gray-box attacks, in which the adversary has partial knowledge of the system – such as the features used or model architecture – but lacks access to the training data and model parameters [1].

In this study, **we focus on black-box attacks** as they better reflect real-world adversarial conditions, where attackers have little to no knowledge of the underlying models.

An additional fundamental challenge lies in preserving the realizability of adversarial traffic, since network protocols impose strict constraints on how traffic may be modified while remaining legitimate. For example, altering header fields can lead to malformed packets that are rejected by routers or hosts. Even when changes are made carefully, they may subtly alter the semantics of the communication, potentially breaking a TCP connection or other protocols. Similarly, excessive perturbations to packet timing or size can render traffic benign or non-functional, preventing it from achieving the intended malicious objective. In such cases, the traffic might still evade the model but fail elsewhere in the network, thereby making the adversarial modifications ineffective or detectable. Balancing these constraints significantly increases the complexity of adversarial traffic generation compared to domains such as computer vision.

The real-time deployment of adversarial traffic aggravates these challenges further. Unlike offline attacks, which give the adversary time to develop and optimize adversarial samples—yet are generally easier to detect and mitigate—, real-time attacks require modifications to be performed on-the-fly, typically under dynamic network conditions. Any additional latency introduced by adversarial sample generation may itself become a signal for detection, emphasizing the need for efficient and low-latency adversarial traffic generation.

1.2 Rationale and Contributions

This thesis introduces TANTGen, a new framework based on the Transformer architecture [15] designed to generate adversarial traffic that adheres to the network protocol constraints. The rationale for choosing this architecture is as follows. Transformers have demonstrated strong capabilities in capturing complex dependencies and contextual relationships in domains such as Natural Language Processing (NLP). Similarly, network traffic exhibits rich temporal and structural dependencies across packets and features, which must be preserved to ensure that the generated traffic remains feasible under protocol constraints. Furthermore, the ability to pre-train Transformer models with large unlabeled datasets is an advantage, given the scarcity of labeled adversarial traffic datasets. By leveraging existing tools for traffic extraction, the Transformer model can learn the underlying structure of traffic patterns without requiring labeled data.

We use TANTGen to analyze the performance of current ML models under adversarial conditions, providing insights into their vulnerabilities. This analysis aims to identify which features and patterns are most susceptible to manipulation by an attacker. Additionally, the self-attention mechanism of Transformers, which allows the model to focus on and weight different parts of the input in parallel, enables efficient processing and potential speed-ups, making the approach more suitable for online or near real-time deployment scenarios.

This thesis makes the following contributions:

- A Transformer-based framework for adversarial traffic generation in the traffic-space domain.
- A black-box threat model that enforces network protocol constraints, ensuring realistic attack scenarios.
- A comprehensive evaluation using real-world datasets and NIDSs, assessing both attack effectiveness and realism.
- An experimental demonstration of effective evasion while preserving traffic semantics. Under a strict $3\times$ flow duration ratio constraint, TANTGen achieves an Attack Success Rate (ASR) of 22.2% compared to 18.1% from TrafficManipulator [1], and under a more relaxed $6\times$ constraint, the gap widens significantly, with TANTGen reaching 99.9% against 75.2%, suggesting that the Transformer architecture is better able to exploit the available perturbation budget.

1.3 Organization of the Document

This thesis is organized as follows. This chapter introduced the research problem, the work objectives and contribution, and the motivation behind generating adversarial network traffic. Chapter 2 provides essential background on Machine Learning concepts, including data preparation and representation, model training, and deployment. It also discusses adversarial attacks and defenses, and presents an overview of existing ML-based networks. Chapter 3 reviews the existing literature on adversarial attacks in ML-based networks, covering both white-box and black-box attacks in feature-space and traffic-space, as well as studies exploring the use of Transformers for networking tasks. Chapter 4 presents the proposed TANTGen framework. This chapter details the threat model, the overall TANTGen architecture, and its implementation. Chapter 5 evaluates our proposal: describes the experimental settings and analyzes the performance of TANTGen in generating adversarial traffic capable of evading current ML models. Finally, Chapter 6 summarizes the main conclusions of the thesis, discusses the limitations of the system, and outlines directions for future work, including potential improvements of the framework.

2

Background

Contents

2.1 Machine Learning (ML) 5
2.2 Adversarial Attacks on Machine Learning 10
2.3 ML-based networks 12
2.4 Summary 15

This chapter provides the necessary background to understand the problem addressed in this thesis, with a particular focus on concepts that are directly relevant to adversarial traffic generation against ML-based networking systems. To this end, we divide the chapter into three main sections: Machine Learning (ML), Adversarial Attacks on Machine Learning, and ML-based networks.

2.1 Machine Learning (ML)

Machine Learning has gained significant popularity due to its ability to achieve high accuracy across a wide range of tasks, making it particularly appealing for practical applications. However, understanding its underlying principles, capabilities, and limitations is essential, particularly when deploying ML models

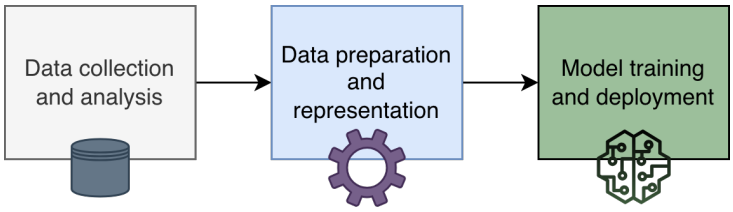


Figure 2.1: Machine Learning pipeline

in adversarial or security-sensitive environments. In Figure 2.1, we present the typical Machine Learning pipeline.

The first step in building a machine learning model is to collect and analyze the data. Exploratory Data Analysis (EDA) can be performed using visualization and statistical techniques to identify trends, correlations, and outliers in the data. This initial analysis informs decisions about how to prepare and transform the data.

In the following sections, we describe the subsequent phases of data preparation and representation, and model training and deployment, covering the procedures necessary to develop an ML model that achieves optimal performance. We also present several well-known model architectures that are relevant to our research.

2.1.1 Data Preparation and Representation

Preprocessing is the first step of data preparation and ensures that raw data is transformed into a suitable numerical format for the model. This may include handling missing values, normalization and standardization, and encoding categorical variables (e.g., One-Hot Encoding, Label Encoding). These operations reduce noise, improve numerical stability, and make the data compatible with the intended model.

Traditionally, feature engineering—applied after preprocessing and before model training—has been an important step in the Machine Learning process. It involves creating, transforming, or combining features to enhance the model's ability to capture patterns and relationships. Well-designed features can also improve interpretability, allowing humans to better understand the model's inner workings and decisions, thereby increasing confidence in its outputs.

With modern deep learning models, including Transformer-based architectures, manual feature engineering is often replaced by automatic representation learning. These models extract rich and task-relevant representations directly from raw data, reducing the need for manual feature design while capturing complex relationships in the input.

Regardless of the approach—manual feature engineering or automatic representation learning—well-constructed representations help reduce data complexity, improve computational efficiency, enable

faster training, and generate models that are less prone to overfitting.

2.1.2 Model Training and Deployment

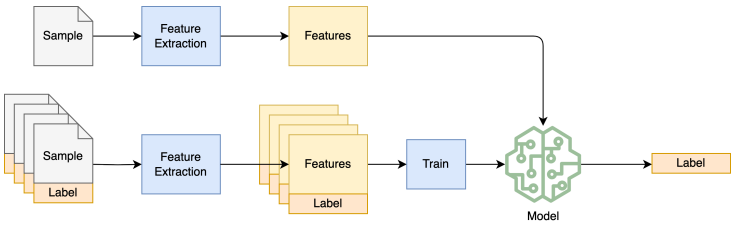
There are several ways to train a machine learning model, which must be chosen taking into account the nature of the task. Training approaches can be divided into three main categories:

1. **Supervised Learning:** In supervised learning, both samples and labels are given to the model for training. This means that the model learns the patterns and relationships between the features extracted from the data and assigns them directly to a label. Within this category we have models such as Decision Tree (DT) [16], Naïve-Bayesian classifier [17], k-Nearest Neighbor classifier [18], Neural Network [19], Perceptron [20], Logistic Regression (LR) [21], and Support Vector Machine (SVM) [22]. Figure 2.2(a) illustrates the supervised learning process.

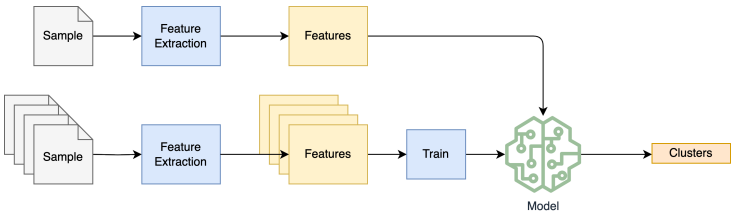
2. **Unsupervised Learning:** On the other hand, in unsupervised learning, only samples are provided to the model. The model learns patterns and relationships between the extracted features, and groups the data without assigning explicit labels to the samples. Some of the models that belong to this category are K-Means Clustering [23], Hidden Markov Model [24], Principal Component Analysis (PCA) [25], Generative Adversarial Network (GAN) [26], and Variational Autoencoder (VAE) [27]. Figure 2.2(b) illustrates the unsupervised learning process.

3. **Reinforcement Learning**: The Reinforcement Learning category is slightly different from the previous ones because, instead of immediately receiving a set of data, it involves an agent that interacts iteratively with the environment. The agent begins by observing some information about the environment and then applies an action. In response, the agent receives the reward for applying that action to that state. The goal of the agent is to maximize the reward. Through this process, the agent learns a mapping between states, actions, and rewards, aiming to maximize long-term cumulative reward. Q-learning [28] is one example of a reinforcement learning algorithm. Figure 2.2(c) illustrates the reinforcement learning process.

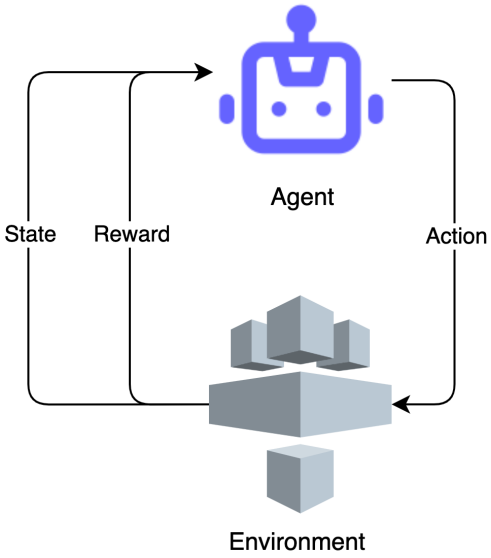
In practice, it is often beneficial to leverage pre-trained models, which have been trained on large, generalized datasets. **Fine-tuning** is the process of adjusting the parameters of these pre-trained models to suit a new, specific task. This approach allows a model trained for one task to be effectively adapted for another, requiring significantly less data and computational resources compared to training from scratch. Fine-tuning can involve updating the entire model or selectively training certain layers while freezing others, depending on the amount of available data and the complexity of the task. This strategy takes advantage of existing information, making it especially beneficial when dealing with limited data or computational resources.



(a) Supervised Learning process



(b) Unsupervised Learning process



(c) Reinforcement Learning process

Figure 2.2: Three basic machine learning paradigms

Although machine learning models are often associated with specific categories of training, some architectures are flexible and applicable to other types of learning. Generative models like GANs and VAEs have demonstrated their usefulness not only in unsupervised learning but also in supervised and reinforcement learning tasks. These models have demonstrated strong capabilities in generating realistic data distributions, making them particularly relevant for adversarial example generation.

Generative Adversarial Networks (GANs) [26] consists of two neural networks that compete against each other in a zero-sum game. The first neural network, called the Generator, produces new data instances, while the second, called the Discriminator, evaluates their authenticity. Specifically, the Discriminator aims to distinguish between real data and data generated by the Generator, whereas the Generator attempts to deceive the Discriminator. Formally, the objective of a GAN can be expressed as:

$$\min_G \max_D \; \mathbb{E}_{x \sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))] \tag{2.1}$$

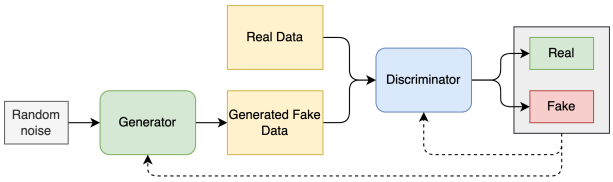
where $D(x)$ is the Discriminator's estimated probability that a real sample x is genuine, $G(z)$ is the Generator's output given a noise vector z drawn from a prior distribution $p_z(z)$, and $p_{\text{data}}(x)$ is the real data distribution. The Discriminator maximizes this objective by correctly classifying real and generated samples, while the Generator minimizes it by producing samples that the Discriminator cannot distinguish from real data. After each iteration, losses are computed for both neural networks and used during backpropagation to update their parameters. This model has been applied in the context of networking, for example, in NetShare [29], to generate synthetic traffic traces that preserve statistical properties of

real traffic. These traces enable the creation of realistic datasets for training and testing network security systems, such as NIDSs, without relying on sensitive or hard-to-obtain real data. Figure 2.3(a) illustrates this architecture.

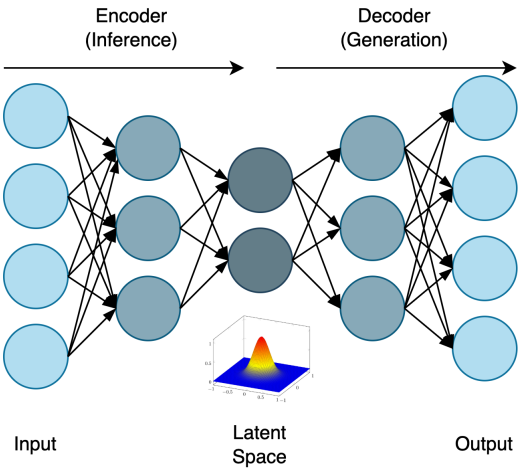
Variational Autoencoders (VAEs) [27] is a type of probabilistic generative model designed to learn efficient representations of data. Like traditional autoencoders, they consist of two main components: an encoder that transforms input data into a compact representation, and a decoder that reconstructs the original data from this encoded representation. However, VAEs introduces the concept of a probabilistic latent space[1], connecting the encoder and decoder in a way that allows for more flexible and powerful data generation. In a VAE, the encoder transforms the input data x into a probabilistic distribution in the latent space, characterized by a mean vector μ and a standard deviation vector σ. These parameters define a Gaussian distribution $q(z|x)$ from which a latent variable z is sampled. The decoder then takes z as input and reconstructs the original data by outputting a distribution $p(x|z)$. This framework enables the model to both encode and generate data while optimizing the variational lower bound, a mathematical objective that balances reconstruction quality and latent space regularization. Figure 2.3(b) illustrates this process.

Finally, the **Transformer** [15] is an architecture based on self-attention mechanisms. While it gained prominence in NLP, its use has been extended to other domains. Self-attention mechanisms model how different parts of the input relate to each other, allowing the model to capture contextual information and long-range dependencies. This capability is particularly important in the context of network traffic, where dependencies across packets and time must be preserved to ensure both realism and protocol compliance. Additionally, Transformers process inputs in parallel, enabling efficient training and making them well-suited for applications that require modeling long sequences. In the context of language modeling, inputs to the Transformer are divided into smaller units called tokens. Tokens can represent words, subwords, or even individual characters, depending on the application. Depending on the architecture, a Transformer may include an encoder, a decoder, or both. The encoder maps the input sequence into contextual representations, while the decoder generates the output sequence based on these representations and previously generated tokens. During decoding, a masking mechanism is applied to the self-attention layer to prevent the model from attending to future tokens. This ensures that each prediction is based only on the tokens generated so far, enforcing the autoregressive property. There are several Transformer configurations tailored to different tasks. Encoder-only models, such as BERT [30], are optimized for tasks like classification, where understanding the full input context is crucial. Decoder-only models, such as GPT [31], are optimized for generative tasks, where the goal is to predict the next token in a sequence. Encoder-decoder models, such as T5 [32] and BART [33], combine both components and are well-suited for tasks like translation and summarization, where both understanding

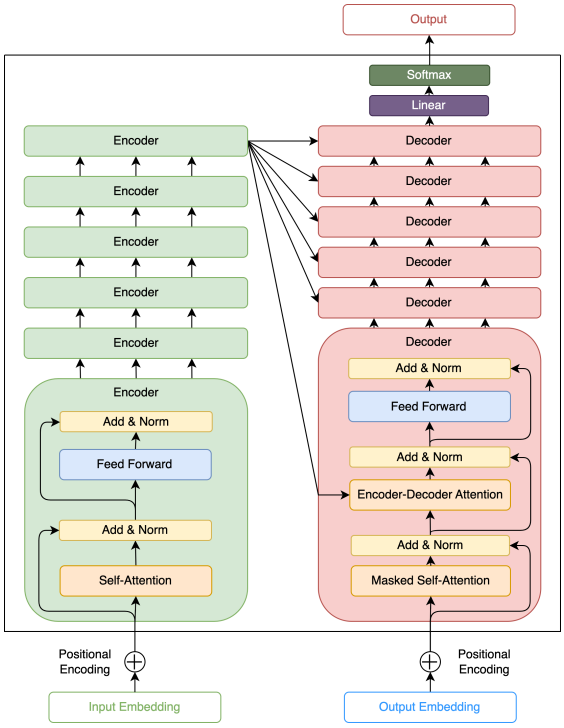
[1] A latent space is a lower-dimensional representation of high-dimensional data. Instead of a deterministic representation, a probabilistic latent space associates each data point with a probability distribution in the latent space.



(a) Generative Adversarial Network architecture



(b) Variational Autoencoder architecture



(c) Transformer architecture

Figure 2.3: Machine Learning Models

and generation are required. Some studies, discussed in Section 3, have already explored the use of Transformers for traffic modeling and generation [34,35], motivating their use in this thesis. Figure 2.3(c) illustrates the general architecture of a Transformer.

2.2 Adversarial Attacks on Machine Learning

Despite its impressive achievements across various domains, the application of ML in increasingly sensitive tasks raises concerns about its vulnerabilities. The purpose of this section is to gain insight into some of these vulnerabilities, specifically concerning adversarial attacks, which are increasingly important today with the widespread use of ML solutions.

In this thesis, we focus on adversarial attacks in which an adversary introduces small, carefully crafted perturbations to the input in order to induce misclassification. The goal of the attacker is to minimize the perturbations to a point that it becomes indiscernible to the human eye or to the monitoring system, thus going unnoticed. In other words, the attacker's goal is to find a perturbation that maximizes the loss of the target model while minimizing the perturbation.

2.2.1 Types of Adversarial Attacks

Adversarial attacks on Machine Learning models may exploit vulnerabilities at different stages of the model lifecycle, including training and deployment. In the training phase, **poisoning attacks** [36–38] inject a small portion of malicious samples into the training set, aiming to degrade accuracy, introduce vulnerabilities, or cause targeted errors. During deployment, **evasion attacks** [1, 6, 9–14] craft inputs to trick the model into misclassifying them. The realizability of these attacks depends on the attackers' capabilities, which also influence the necessary defense mechanisms.

Adversarial attacks can be categorized based on the attacker's knowledge of the target system, another important distinction for designing effective defenses. **White-box attacks** [6, 8, 10, 13, 14] occur when the attacker has complete knowledge of the target model, including its architecture, parameters, and training data. Although this is not always the case, it can be claimed that security through obscurity is not a sound strategy from the defender's perspective and that the defender must, therefore, be prepared for the case in which the attacker has complete model information. As expected, white-box attacks typically achieve higher success rates, but their assumptions are often unrealistic in practical settings. In contrast, **gray-box attacks** [1] involve partial knowledge, where the attacker usually has access to the model architecture or features used, but not to its training set nor parameters. Finally, **black-box attacks** [1, 7, 9–12, 39] assume the attacker has no direct knowledge of the target model. This is a more realistic and dangerous approach as the attacker needs less knowledge to exploit the model's vulnerabilities. These attacks are further divided into two types. In **black-box query-based attacks** [7, 12], the attacker iteratively queries the target model and observes its predictions. By adjusting the inputs based on the observed outputs, the attacker gathers enough information to infer decision boundaries or weak points in the model. This enables the crafting of adversarial examples that can mislead the model without requiring prior knowledge of its structure or parameters. In **black-box transfer-based attacks** [1, 39], the attacker creates and trains a surrogate model, using a collected surrogate dataset, and uses it as if it were the target model. Adversarial examples tested against this surrogate model are then used to attack the target. This is possible thanks to adversarial example transferability, a crucial property of adversarial attacks, first observed by Szegedy et al. [40]. This property refers to the ability to generalize between models. This means that adversarial examples built against one model often remain effective against others, with possibly lower success rates, even if the models differ in architecture or training data. In some recent studies [9–11], query and transfer strategies are often combined, using the query outputs to create the surrogate dataset.

2.2.2 Defenses against Adversarial Attacks

Over the past decade, a growing awareness of the vulnerabilities of many ML models to adversarial attacks has prompted extensive research into a variety of defense techniques. This section summarizes key defense mechanisms proposed to improve the robustness of ML models against adversarial attacks.

We start with **adversarial training**, one of the best known and studied techniques [40, 41]. In this approach, models are exposed to adversarial examples during the training process. This enables models to recognize and resist adversarial perturbations, enhancing their robustness against these attacks.

An essential step in enhancing model robustness involves **designing and adjusting input features**. Although the goal during feature engineering is to create features robust to adversarial perturbations, if this robustness does not hold, it is important to modify the features, for instance through insertion, deletion, and resizing [42, 43]. These adjustments help reduce the attack surface and often boost model performance in adversarial scenarios.

Targeting the model input directly, **sanitization** identifies and eliminates maliciously perturbed inputs early in the process. This sanitization can also be performed in training datasets to ensure that potentially dangerous inputs are removed before they can affect the model [44, 45]. Additionally, **preprocessing** techniques such as normalizing inputs to mitigate adversarial disturbances can also help protect models. **Detecting and rejecting inputs far from the training data** in the feature space can also be used as a defense since it can identify and mitigate blind-spot evasion points [46].

Another powerful defense is **defensive distillation**. This method smooths the decision surface of a model in adversarial directions, making it harder for attackers to exploit subtle weaknesses in decision boundaries [39]. It operates in two stages: first, a teacher network is trained on the original dataset, and its output probabilities (soft labels) are extracted; then, a student network is trained to mimic these soft targets instead of hard labels. By learning from the richer information in soft probabilities, which capture interclass relationships, the student network develops more generalized decision boundaries, increasing robustness against adversarial examples.

Finally, using **ensemble models** has also proven to be a good strategy against adversarial attacks [47, 48]. This enhances generalization by leveraging the strengths of different models, thereby mitigating the impact of individual model biases or weaknesses.

2.3 ML-based networks

In this section, we review applications of Machine Learning in networking, with an emphasis on network security, which is the primary focus of this thesis.

Before discussing existing work, it is important to distinguish between the main types of features extracted from network traffic, as this distinction is central to adversarial attack design. **Packet-based**

features are extracted from the individual packets. These features are useful for applications requiring precise information about packet-level behavior, such as protocol analysis. Examples of these features are packet size, header information, payload content, and inter-arrival time. **Flow-based features**, on the other hand, are taken from groups of packets that share characteristics, such as the same 5-tuple (protocol, source and destination IP, source and destination port). Using these features, the analyzer can understand the characteristics of the interaction between two endpoints, for instance, thus being able to identify patterns and anomalies over sessions rather than individual packets. Examples of these features are the duration of the flow and the number of packets or bytes exchanged.

ML has been applied in various areas of networking, like traffic prediction, traffic classification, routing, congestion control, resource and fault management, Quality of Service (QoS) and Quality of Experience (QoE) management, and network security [2].

ML is used to **predict future network traffic**, enabling optimized resource allocation and improved performance. For example, Yu et al. [49] were the first to apply Machine Learning Perceptrons (MLPs) to predict traffic patterns, offering an improvement over traditional autoregressive models.

In the field of **traffic classification**, ML has been used to categorize network traffic to improve quality of service and security, also playing a significant role in, for example, censoring network traffic. Network traffic censorship is used by governments and control authorities to restrict access to online information or platforms. It may involve techniques such as blocking websites or IP addresses, keyword-based content filtering, and deep packet inspection. Although tools such as VPNs are already used to bypass these restrictions, some features are not hidden by these tools, which makes them not always effective. Therefore, new approaches have been studied for circumventing censorship systems [7, 12]. The Great Firewall of China serves as a prominent example of network traffic censorship, employing advanced ML techniques to detect and block censored content. Nguyen et al. [50] introduced a flow-based ML classifier for internet traffic classification, demonstrating its effectiveness in distinguishing various traffic types by leveraging limited transport layer information. The approach focuses on utilizing statistical features of sub-flows, derived from a small sliding window of packets, and employs (unsupervised) clustering techniques alongside supervised learning methods to achieve high accuracy.

Traffic routing optimization benefits from ML by dynamically identifying the best paths in networks to minimize latency and improve throughput. Several investigations focus on the use of Reinforcement Learning for this purpose, namely Forster et al. [51] that used a Q-learning approach for Wireless Sensor Networks to efficiently send data from one source to multiple mobile receivers, called FROMS (Feedback Routing for Optimizing Multiple Sinks). It works by learning and choosing the best shared paths to minimize the number of hops, or steps, the data takes.

ML application for **congestion control** helps manage network bottlenecks, ensuring network stability, fairness in resource utilization, and acceptable packet loss rate, thus improving overall service

quality. For instance, Liu et al. [52] used a ML approach to identify the cause of packet loss in hybrid wired-wireless networks, distinguishing between losses due to congestion and those caused by wireless transmission errors. Their method demonstrated greater flexibility and accuracy compared to traditional techniques, though it relied on the assumption of static Round-Trip Time (RTT) values, which can be unrealistic in dynamic networks.

Resource and fault management, another vital area, relies on ML for efficient allocation and automatic fault detection, isolation, and correction. For instance, Pellegrini et al. [53] proposed an ML-based framework to predict the Remaining Time To Failure (RTTF) of applications in a way that is adaptable to various scenarios. The framework uses different ML techniques for building prediction models, thus selecting the most suitable technique based on the needs. While the framework's accuracy improves as failure approaches due to accumulating anomalies, it has higher prediction errors when failures are still far off.

To improve **QoS and QoE management**, ML has also been proposed to ensure networks meet user expectations and maintain service agreements. User experience is measured in terms of network parameters (e.g., bandwidth, packet loss rate, delay, jitter) and application parameters, such as bitrate for multimedia services. Mushtaq et al. [54] used ML classifiers, such as Decision Trees (DTs) and Random Forests (RFs), to model the relationship between QoS and QoE. Their approach analyzed video streaming in controlled network conditions, using features like delay, jitter, packet loss, and video resolution. DTs and RFs achieved good results, but the practical relevance of some viewer-related features remains unclear for real-world applications.

Lastly, ML plays a crucial role in **network security**, particularly in detecting and mitigating threats like intrusion and Denial-of-Service (DoS) attacks. **Network Intrusion Detection System (NIDS)**, for example, inspect packets and flows to detect malicious activity in network traffic. These systems are usually categorized into signature-based NIDS and anomaly-based NIDS. **Signature-based NIDS** compares network activity patterns with already known attacks to identify similar behaviors. While effective at detecting known attacks, these systems cannot identify zero-day attacks and require frequent updates to remain effective. In contrast, **anomaly-based NIDS** focuses on identifying deviations from normal traffic behaviors, making them capable of detecting zero-day attacks. However, this approach often results in higher false positive rates since not all normal traffic behaviors can be accurately represented in a dataset. For instance, Kitsune [55] uses an ensemble of autoencoders trained on benign traffic to identify anomalies in real-time by measuring reconstruction loss, where malicious packets typically exhibit higher reconstruction loss compared to benign packets. This makes it effective in detecting unknown threats.

2.4 Summary

This chapter reviewed the fundamentals of ML, adversarial attacks, and their application in networked systems. We highlighted key model architectures, particularly Transformers, due to their ability to capture complex dependencies in sequential data. We also discussed adversarial threat models, emphasizing black-box attacks as the most realistic setting, and outlined existing defense mechanisms. Finally, we described a variety of ML-based network applications.

3

Related Work

Contents

3.1 Adversarial Attacks in ML-based Networks 17
3.2 Use of Transformers for Networking Tasks 23
3.3 Summary 24

This chapter reviews the state-of-the-art on adversarial attacks in ML-based network systems and related uses of Transformer architectures. These works provide the foundation for our design and highlight the limitations that motivate TANTGen.

3.1 Adversarial Attacks in ML-based Networks

Adversarial attacks in ML-based networks can be broadly categorized along two orthogonal dimensions: (i) the space in which the attack is performed (feature-space vs. traffic-space), and (ii) the attacker's knowledge of the target model (white-box vs. black-box). **Feature-space attacks** directly manipulate the input features of ML models to avoid detection. While such changes can be highly effective, it is unrealistic to assume that the attacker can directly modify the input features. Furthermore, unlike in fields such as image recognition, the feature extraction procedure results in a loss of information about

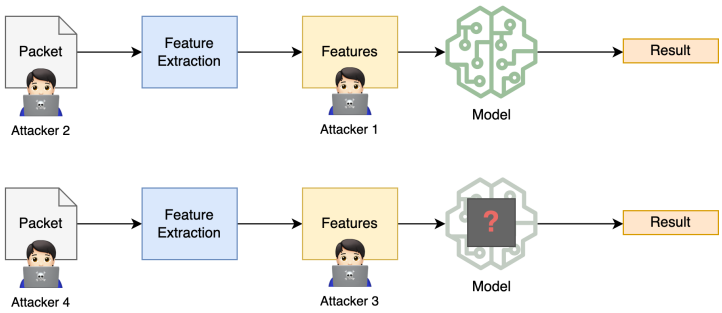


Figure 3.1: Attacker types

the original packets from where the feature value originated, making it difficult to uniquely or accurately reverse-engineer the original traffic patterns from the derived features. This challenge, known as the inverse feature mapping problem, is a central limitation of feature-space attacks in networking scenarios. In contrast, **traffic-space attacks** consider practical constraints, making modifications directly to the packets, typically in timing and size, while also checking for compliance with network protocols. However, excessive modifications can cause the traffic to lose its maliciousness or appear inconsistent with normal network behaviors. Overall, the networking domain introduces strict semantic and protocol constraints, significantly reducing the space of feasible adversarial modifications compared to other domains such as computer vision.

In Figure 3.1 we illustrate the different attacker types, based on available knowledge about the attacked model and in which space the modifications are applied.

3.1.1 White-box Feature-space attack

In Figure 3.1, this is the attack performed by attacker 1. The white-box feature-space attack is the one that makes the least realistic assumptions. Here, the adversary is positioned after feature extraction, modifying and sending the features directly to the target model. Despite their lack of realism, this research line is still important to allow the understanding of which features the model is vulnerable to.

B.-E. Zolbayar et al. [10] used a Wasserstein Generative Adversarial Network (WGAN) to generate adversarial traffic flows to attack NIDSs, proposing an approach named NIDSGAN. In this GAN, the generator aims to effectively mutate the traffic, creating a balance where the perturbations are minimal, the modified traffic is classified as benign by the NIDS, and it remains indistinguishable from real attack flows. The discriminator is used to enforce this last requirement by distinguishing between original attack samples and the generated adversarial flows. WGANs provide more stable training than standard GANs by using the Wasserstein distance to measure the difference between generated and real samples. Instead of performing a binary classification, the discriminator (or critic) performs a regression task by outputting a continuous score that reflects the realism of the input. To maintain traffic realizability, the

authors enforce inherent constraints and valid feature ranges.

P. Venkata et al. [13] used VAEs to reconstruct malicious packets, also studying and comparing several variations of this model. Typically using multilayer perceptrons, VAEs have also been tested with Wavelet Neural Networks, which use scaling and translation operations to create new wavelet functions[1] from a base wavelet function. These wavelet functions are applied to the input data, allowing the network to focus on capturing multi-scale features and complex patterns of the model effectively. This technique generally allows a better approximation of non-linear functions and faster model training. In the latent space, the use of chaotic maps[2] was tested. The goal of integrating these maps was to increase the diversity of adversarial samples, introducing sensitive and apparently random patterns.

3.1.2 White-box Traffic-space attack

This is the attack represented by attacker 2 in the previous figure. In a White-box Traffic-space attack the network packets are modified directly before the feature extraction phase. This approach is more realistic as the attacker does not need to have access to the pipeline server where the feature extraction module is running, but instead, it is only able to modify real network traffic.

Targeting NIDSs, M. J. Hashemi et al. [6] propose applying three simple mutations to packets. First, malicious packets are split into smaller packets while performing the necessary updates to ensure the attack remains effective, which helps evade detection by altering packet sizes and breaking recognizable attack patterns. Second, the time intervals between packet transmissions are adjusted to mimic benign traffic patterns while ensuring that no timeouts occur. Third, extra packets – processed by the NIDS but with no additional impact on the target system – are injected. To derive realistic flow-based features, these are separated into groups according to their dependencies, and masks are defined to block updates to specific features based on the groupings. The attacker iteratively applies these transformations (Split, Delay, or Inject) to reduce the probability of being classified as malicious traffic by staying below the detection threshold. However, this iterative process is computationally expensive and unsuitable for real-time deployment.

F. Pierazzi et al. [8] were the first to introduce the formalization of problem-space attacks (which in the context of our problem would be traffic-space attacks). Although their work targets malware classifiers rather than NIDSs, the conceptual framework they propose is broadly applicable and relevant to our problem. They define the available transformations (based on addition, removal, or modification) that are allowed in the problem-space while preserving semantics, plausibility, and robustness to preprocessing. Having semantics preserved means that the changes do not alter the essential behavior or intent of the object. Plausibility, in turn, ensures that the transformed object appears legitimate or realistic upon

[1] A wavelet function is a wave-like oscillation that starts at zero, fluctuates by increasing or decreasing, and then returns to zero once or multiple times.

[2] Chaotic maps are mathematical systems known for their deterministic but unpredictable behavior.

manual inspection. Finally, robustness to preprocessing prevents detection, i.e., it ensures that adversarial examples support typical preprocessing techniques (e.g., dead code removal in software or noise reduction in images) that can stop adversarial perturbations. The paper then demonstrates the application of this formalization by fooling a state-of-the-art malware classifier with Android malware. The approach used focuses on injecting benign code, so that malicious intentions are camouflaged while keeping them functional. Here, semantics are preserved by surrounding the benign code with conditional instructions whose result is always False. Therefore, robustness to preprocessing is maintained by making these conditions difficult to calculate in static analysis, making it necessary to calculate them at runtime. Finally, the plausibility is maintained by using benign code snippets from real applications.

M. Jin et al. [14] proposed a framework, called PANTS, that combines Adversarial Machine Learning (AML) techniques to generate adversarial features, with Satisfiability Modulo Theories solvers to find packet sequences that do not diverge too much from the AML's output but are realizable and consistent with the threat model and domain-specific constraints. These packet sequences are then used to improve the robustness of ML-powered network applications against realistic adversarial attacks through adversarial training. The framework considers two attacker positions: end host attackers, who can delay packets, inject new packets, and append dummy payload to existing packets; and in-path attackers, who are restricted to delaying packets only, as injecting packets and appending dummy payload would be detectable from the application perspective. The AML techniques used are Projected Gradient Descent (PGD) or Zeroth-Order Optimization (ZOO) that perturb features to maximize loss. PANTS overcomes limitations in existing tools, such as the inverse feature mapping problem and the instability in reinforcement learning-based approaches such as Amoeba [56] (which will be explained in section 3.1.4). However, its iterative process is also time-consuming and cannot be performed in real-time.

3.1.3 Black-box Feature-space attack

We now turn to attacker 3 in Figure 3.1. Black-box feature-space attacks inherit the limitations of feature-space approaches (notably inverse feature mapping) while operating under more realistic threat assumptions, where the attacker doesn't know the model being attacked. Typically, training and testing are usually performed with a surrogate model that may not be the same as the target model. To create this surrogate model, the most commonly used approaches usually combine queries and transferability. In short, queries are sent to the target model, which are combined to create a dataset. This dataset is then used to train the surrogate model. It is then assumed that attacks that fool this surrogate model will also fool the target model, considering that the attack is transferable.

P. T. Duy et al. [9] propose a framework to deceive NIDSs using WGAN and function-preserving, called DIGFuPAS. To address the black-box model problem, a surrogate model is created, trained from the dataset created by the target model's outputs to queries. The framework emphasizes function-

preserving feature manipulation by distinguishing between functional features, which are essential for the primary behavior and intent of the data flow, and non-functional features, which do not impact the attack's functionality. By keeping the functional features unchanged, this approach increases the effectiveness of adversarial samples in evading detection. Moreover, DIGFuPAS also provides feedback for adversarial packets to be used by the NIDS for adversarial training and robustifying.

B.-E. Zolbayar et al. [10], already mentioned in the white-box attacks section, also propose a solution for black-box attacks. The paper combines its solution for white-box attacks, using WGAN to generate adversarial traffic, with the creation of a surrogate model, also trained from the dataset created by the outputs of the target model for queries. In addition to the black-box threat model, a restricted black-box threat model is introduced, where the number of samples available to the adversary is strictly limited. Finally, it proposes an active learning approach to reduce the initial training dataset size of the local model or improve the attack success rates with the same training dataset, by iteratively probing the NIDS model with adversarial samples near its decision boundary and refining the local model.

3.1.4 Black-box Traffic-space attack

Finally, attacker 4 (Figure 3.1) performs the Black-box Traffic-space attack, where no prior knowledge about the attacked model is needed and it operate directly on network traffic. Attacker 4 represents the most realistic and dangerous type of attacker.

D. Han et al. [1] used GANs to generate adversarial features, and a Particle Swarm Optimization (PSO) algorithm to mutate network traffic while maintaining malicious functionality. Inside the GAN architecture, the generator is a feed-forward neural network whose aim is to transform a malicious feature into its adversarial version, while the discriminator is used as a surrogate model of the target one to distinguish generated features from benign ones. The PSO then takes the adversarial features generated by the GAN and determines how to modify the actual network traffic to produce those features while adhering to constraints on overhead, timing, and functionality, by either adding delays or injecting new packets with adjusted inter-arrival times, protocol layers, or packet sizes. Since these transformations do not alter the original packets directly, semantic preservation is more naturally guaranteed. The paper finally proposes an explanation method that identifies the most vulnerable features and then uses adversarial feature reduction to improve robustness by removing those weak features. A direct comparison between TANTGen and this approach is presented in Chapter 5.

P. Sun et al. [11] created GPMT, an adversarial attack strategy that modifies the temporal-spatial features of network traffic, such as delaying the sending time of traffic packets and increasing packet lengths, which can be easily mapped to traffic space. The process begins with a probing phase, in which queries are sent to the target model to collect its responses, which are then used to train a surrogate model. It is then assumed that adversarial examples that fool the surrogate model will also fool the target

model, relying on the transferability property. Then, in the attack execution phase, adversarial traffic is generated using a WGAN by adjusting packet timestamps and payload lengths. The surrogate model is used as the discriminator while the generator creates the packets to deceive the model. To ensure that packets maintain maliciousness and protocol compliance, the modifications applied are limited. However, the magnitude constraints applied do not explicitly account for protocol-specific restrictions at the application layer, where increasing payload sizes may alter fixed-length fields or violate rigid message formats, potentially producing malformed packets depending on the protocol in use.

One limitation shared by both aforementioned works, which rely on GANs, is their inability to effectively capture long-range dependencies, as they typically focus on individual packet features or aggregate flow features. This limitation motivates the use of Transformer architectures, which are inherently designed to model sequential dependencies and can better capture the temporal structure of network traffic—an intuition that may find some reflection in our empirical findings presented in Chapter 5, as the Transformer-based model outperformed the approach proposed by D. Han et al. [1], though other factors may also contribute to this performance gap.

The following two papers, while originating from the censorship evasion domain rather than intrusion detection, are closely aligned with our research. In censorship evasion, the goal is to disguise network traffic to bypass classifiers that enforce content or access restrictions — a fundamentally similar challenge to adversarial attacks on NIDSs, where traffic must evade classifiers designed to detect malicious activity. Both domains share the core technical objective of fooling a network traffic classifier, making these works a relevant source of insights.

K. Bock et al. [7] propose Geneva, a new way of automating the creation of censorship evasion strategies. The process is initialized by creating a population of random “forests”, i.e., collections of packet manipulation trees. Each tree has triggers and actions such as duplicating, fragmenting, tampering, or dropping packets. Geneva then evaluates each forest by testing it against a censor and scoring based on connection success. Forests that demonstrate good performance are maintained, and the worst are discarded. Then, the mutation phase introduces random changes to the trees, such as changing triggers or actions, so that the algorithm can explore new strategies. The crossover phase is then used to merge the trees and forests that showed the best results, in order to produce new combinations. This iterative process continues over multiple generations, evolving increasingly effective strategies. Once again, being an iterative process, it can be time-consuming. Moreover, by directly sending attack attempts to the target model, Geneva is more likely to be detected and blocked.

H. Liu et al. [12] created Amoeba, an agent trained with reinforcement learning to perturb each packet to maximize the likelihood that the altered flow will deceive network censorship. The perturbations applied to the packets are truncation, padding, and delay, and several restrictions are formalized (such as packet order and inter-packet delays) so that the model applies mutations in compliance with these

restrictions, thus ensuring that the protocol rules are not broken. It operates in a black-box setting, giving a reward to the agent based on the output of the censorship attempt. To ensure that Amoeba is robust and flexible in different network environments, different packet loss rates and highly noisy rewards were tested. Nevertheless, Amoeba presents several limitations: as noted by M. Jin et al. [14], in addition to instability during training, it tends to exploit a single vulnerability rather than exploring new ones, making it less diverse and less effective for robustifying classifiers through adversarial training.

3.2 Use of Transformers for Networking Tasks

Transformers are powerful sequence modeling architectures that can be adapted to a diverse range of applications. When pre-trained, they can be adapted to a wide range of tasks with little labeled data, achieving surprising results. For this reason, several studies have been carried out to create transformers that can be fine-tuned for various networking tasks, such as traffic classification and attack detection.

Franck Le et al. [57] examine the potential of applying foundational models such as BERT and GPT-3 to network traffic analysis and management. The authors note that network traffic shares structural similarities with natural language, and that many downstream tasks in networking—such as packet classification, anomaly detection, and traffic generation—rely on machine learning approaches like classification, generative modeling, and reinforcement learning. These same approaches have been successfully applied or are being actively explored in NLP, suggesting that foundational models could similarly improve performance, generalization, and scalability in network traffic tasks. The abundance of unlabeled data and the rich semantic structure of network traffic further support pre-training foundational models for these tasks.

Dietmüller et al. [34] presented Network Traffic Transformer (NTT), an adaptation of the encoder-only transformer architecture designed for modeling network traffic dynamics. The authors incorporate packet-level features such as timestamps, sizes and delays into embeddings. They then employ hierarchical aggregation to manage long-term and short-term dependencies. During pre-training, the goal of the transformer is to predict the end-to-end delay, allowing the model to then generalize to multiple tasks. NTT is divided into several layers that, during training, help it learn which features are useful (embedding layer), how to aggregate them over time (aggregation layer), and infer the context of the entire sequence (transformer encoder layers). It then has a replaceable decoder that is updated or replaced for the specific task during the fine-tuning.

Jiawei Zhou et al. [35] proposed NetFlowGen, a decoder-only transformer pre-trained with the objective of predicting the next step of the network traffic. The goal is to capture dependencies and produce general traffic representations for downstream tasks. The data for the pre-training is collected

from NetFlow records, and they then demonstrated fine-tuning to a real-world Distributed Denial-of-Service (DDoS) attack detection task, showing the model's ability to adapt with minimal task-specific labeled data. An important specificity to be highlighted is that the authors chose to discretize the continuous features before inserting them into the model. They argue that one of the obstacles one may encounter in the network traffic domain is that packet features may contain continuous values, such as the number of packets or bytes. This represents a mismatch when compared with the typical NLP applications of the Transformer, which can be a problem. Effectively, the transformer architecture is designed to process languages that contain a finite vocabulary, thus making it possible to interpret words as discrete values. To address this mismatch, the authors then present a solution to this problem using a binning algorithm. More specifically, they collect all the values that appear in the training dataset for each feature and divide that range of values into bins. Each continuous value then corresponds to a discrete value (the corresponding bin number). Finally, they transform the discretized time and traffic features into one-hot encodings and concatenate them.

Motivated and inspired by the recent application of the Transformer architecture to networking tasks, and recognizing its relevance to our problem domain, this thesis aims to leverage these models to generate adversarial traffic and, in turn, enhance the robustness of ML-based networking applications. Unlike prior work, which focuses on traffic modeling or classification, we employ Transformer architectures for adversarial traffic generation under realistic constraints, combining (i) a black-box threat model, (ii) traffic-space modifications, and (iii) protocol-compliant generation of network traffic.

3.3 Summary

In this chapter, we reviewed several studies, including works on generating adversarial attacks against ML-based networks and others applying Transformer architectures to networking tasks. However, to the best of our knowledge, no prior work has combined the use of Transformers for generating adversarial attacks under realistic network constraints. Table 3.1 summarizes the key characteristics of existing approaches, highlighting the gap addressed by this thesis.

Our goal is to generate adversarial network traffic under a realistic threat model and network assumptions. Specifically, we focus on black-box attacks that introduce subtle modifications to network traffic while preserving its semantics and respecting practical constraints of the traffic space. Building on recent advances in sequence modeling, we propose to apply Transformer architectures to this problem domain.

Overall, existing approaches either rely on unrealistic assumptions (feature-space or white-box), fail to preserve traffic semantics, or struggle to capture long-range dependencies. These limitations motivate the design of TANTGen.

Table 3.1: Comparison between existing research

Reference	Year	Attack Type	Attack Scope	Model
B.-E. Zolbayar et al. [10]	2022	❍ & ●	✤	WGAN
P. Venkata et al. [13]	2023	❍	✤	VAE w/WNN
M. J. Hashemi et al. [6]	2019	❍	🌐	Own algorithms
F. Pierazzi et al. [8]	2020	❍	🌐	Own algorithms
M. Jin et al. [14]	2024	❍	🌐	PGD & ZOO & SMT
P. T. Duy et al. [9]	2021	●	✤	WGAN & function-preserving
K. Bock et al. [7]	2019	●	🌐	Own algorithms
H. Liu et al. [12]	2023	●	🌐	Reinforcement Learning
D. Han et al. [1]	2021	●	🌐	GAN & PSO
P. Sun et al. [11]	2023	●	🌐	WGAN
Our thesis	2025	●	🌐	Transformer

Legend: ❍ White-box attack, ● Black-box attack, ✤ Feature-space scope, 🌐 Traffic-space scope
GAN - Generative Adversarial Network ; WGAN - Wasserstein Generative Adversarial Network ; VAE - Variational Autoencoder ; WNN - Wavelet Neural Networks ; PGD - Projected Gradient Descent ; ZOO - Zeroth-order Optimization ; SMT - Satisfiability Modulo Theories ; PSO - Particle Swarm Optimization

4

Solution

Contents

4.1 Threat model 27
4.2 TANTGen Architecture Overview 28
4.3 Implementation 35
4.4 Summary 36

In this chapter, we propose TANTGen, a novel Transformer-based architecture for generating adversarial traffic aimed at fooling ML-based networks, with a particular focus on Network Intrusion Detection Systems (NIDSs). We present the Threat model, the TANTGen Architecture Overview, and its implementation.

4.1 Threat model

The considered threat model addresses evasion attacks on ML-based NIDS operating in the network traffic domain. Evasion attacks occur at inference time, where the adversary crafts inputs to avoid detection by an already deployed model, in contrast to poisoning attacks, which target the training phase by corrupting the training data. The adversary's objective is to introduce minimal modifications directly

to the network packets (i.e., by manipulating observable traffic characteristics such as timing and size), thus being a traffic-space attack, generating adversarial examples that cause the model to misclassify malicious traffic as benign. Importantly, unlike feature-space attacks that assume direct access to and manipulation of the detector's input features, we consider an adversary that operates on real network traffic without access to the internal feature extraction pipeline of the NIDS. In a practical setting, the attacker could first generate genuinely malicious traffic (e.g., an exploit or scanning activity) and subsequently perturb this traffic to produce adversarial variants that evade detection while preserving the attack's effectiveness. We assume a black-box setting, where the adversary has no knowledge of the model's architecture, parameters, or training data. We further assume an end-host adversary (e.g., a controlled client) capable of generating and shaping its own traffic, enabling fine-grained control over packet timing and payload structure, but without the ability to modify in-transit traffic (i.e., no man-in-the-middle capabilities).

The adversary is allowed to manipulate temporal and spatial traffic features. Temporal manipulations (e.g., inter-packet delays) must remain small, as excessive delays can degrade attack effectiveness or raise suspicion, and may interfere with protocol dynamics (e.g., TCP retransmission timeouts, congestion control behavior) or application-level constraints (e.g., latency-sensitive services such as interactive or video applications). Spatial manipulations (e.g., padding) must preserve protocol correctness, requiring consistent updates to dependent fields such as length indicators, checksums, and sequence-related metadata. Although these transformations are defined over packet-level features (e.g., size and inter-arrival time), they are always mapped back to valid packet-level traffic, ensuring that generated adversarial examples are realizable in practice. Padding can still be applied in encrypted traffic (e.g., via TLS record padding or by adding application-layer bytes prior to encryption), but it is more constrained, as the adversary cannot arbitrarily modify ciphertext without invalidating integrity checks (e.g., MACs); thus, perturbations must be introduced before encryption or through protocol-compliant padding mechanisms. Nevertheless, even when payloads are encrypted, traffic analysis features such as packet sizes and timings remain observable, making them viable targets for adversarial manipulation. Additionally, these modifications are constrained to preserve both the malicious functionality and protocol compliance of the traffic.

Under these constraints, the adversary seeks to achieve successful evasion with minimal perturbation cost.

4.2 TANTGen Architecture Overview

This section presents the proposed transformer-based architecture for generating adversarial network traffic, covering the data representation and problem formulation, the rationale behind model selection,

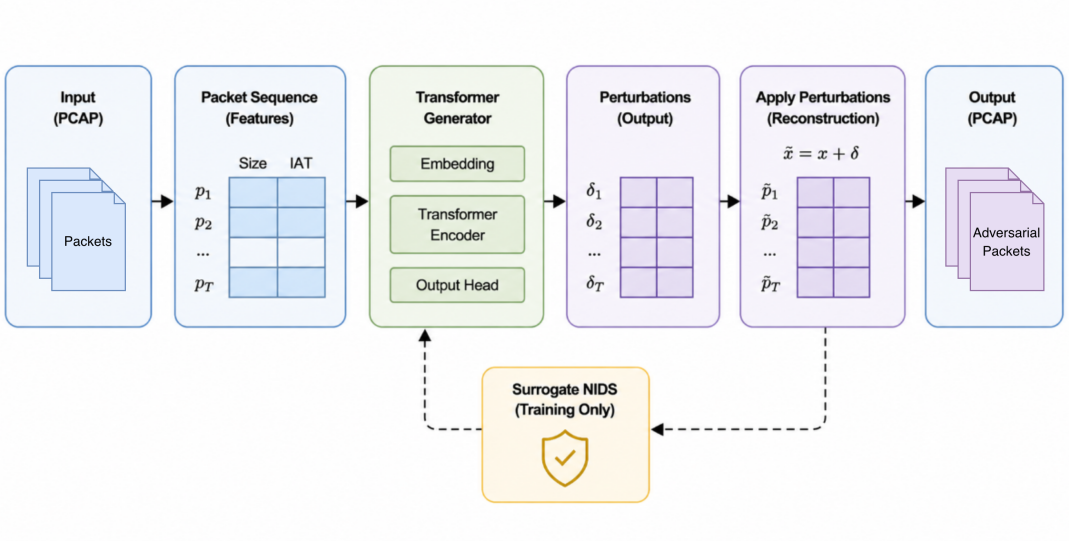


Figure 4.1: TANTGen Overview

the training methodology and the attack application process.

Figure 4.1 provides a high-level overview of the full architecture, illustrating the main components and their interactions, which will be detailed in the following subsections.

4.2.1 Data Representation and Problem Formulation

Choosing an appropriate data representation and problem formulation strategy is a crucial step, as the model's effectiveness depends heavily on how the input is structured. In our case, ensuring a meaningful representation of network packets is particularly important, since the model directly operates on sequential packet-level information. Moreover, the representation must strike a balance between expressiveness and tractability, capturing key traffic dynamics while remaining amenable to efficient learning and manipulation.

Inspired by prior work such as Amoeba [12] and PANTS [14], we represent traffic as a sequence of packets based on two fundamental attributes: *packet size* and *Inter-Arrival Time (IAT)*. These features capture the observable characteristics commonly exploited by traffic analysis-based NIDS, even in encrypted settings. Formally, a flow is represented as a sequence $[p_1, p_2, p_3, \dots]$, where each packet p_t is defined as a two-dimensional feature vector:

$$p_t = [\text{Packet Size}_t, \text{IAT}_t].$$

In line with the "raw packet representation" paradigm of NTT [34], preprocessing is intentionally kept minimal to preserve the fidelity of the original signal. However, lightweight normalization steps are applied to improve numerical stability and training efficiency.

First, IAT values are clipped to be non-negative and transformed using a logarithmic function ($\log(1+x)$). This transformation reduces the skewness of the distribution, meaning it compresses large values and makes the data more balanced. This is particularly important for network traffic, where inter-arrival times typically exhibit heavy-tailed distributions [58, 59].

After this transformation, both packet size and IAT are standardized using the z-score:

$$x' = \frac{x - \mu}{\sigma},$$

where μ and σ are the mean and standard deviation computed from the training data. These values are saved and reused to ensure consistent preprocessing during training and evaluation.

To facilitate the capture of temporal patterns, we partition each sequence into fixed-length windows, using a sliding window mechanism with a configurable stride. Is relevant to note that during the attack application phase, the stride is set equal to the window size, ensuring non-overlapping windows and producing exactly one output per packet. Given a window length L and a stride s, the dataset is transformed into a set of subsequences, each containing L consecutive packets. This sliding-window formulation enables the model to capture both short-term dependencies and longer-range temporal structure while maintaining a fixed input size.

The resulting sequences are then fed into the model, where each packet is projected into a vector embedding[1] via a learnable linear projection layer. This layer maps the low-dimensional input features into a higher-dimensional latent space, enabling the model to capture richer representations between packet attributes. This approach avoids complex feature engineering while preserving the original data structure.

In addition to the input representation, it is important to define the expected output of the model. Given an input sequence of packets, the model produces a sequence of perturbation vectors of the same length. Each perturbation is applied to the corresponding packet features, namely packet size and IAT. Formally, for an input sequence $[p_1, p_2, \ldots, p_T]$, the model outputs a sequence $[\delta_1, \delta_2, \ldots, \delta_T]$, where each δ_t is defined as:

$$\delta_t = [\Delta\text{Packet Size}_t,\ \Delta\text{IAT}_t].$$

These perturbations are then applied to the original packet sequence to obtain a modified version $\tilde{x}$, which is used to evaluate the response of the surrogate NIDS. This formulation allows the model to learn how to minimally adjust traffic features while preserving the overall structure of the sequence.

Figure 4.2 provides an overview of the data representation pipeline.

Building on this formulation, we now discuss the choice of model architecture.

[1] Vector embeddings are numerical representations of data in a continuous high-dimensional space, where semantic similarity between concepts is preserved as geometric proximity.

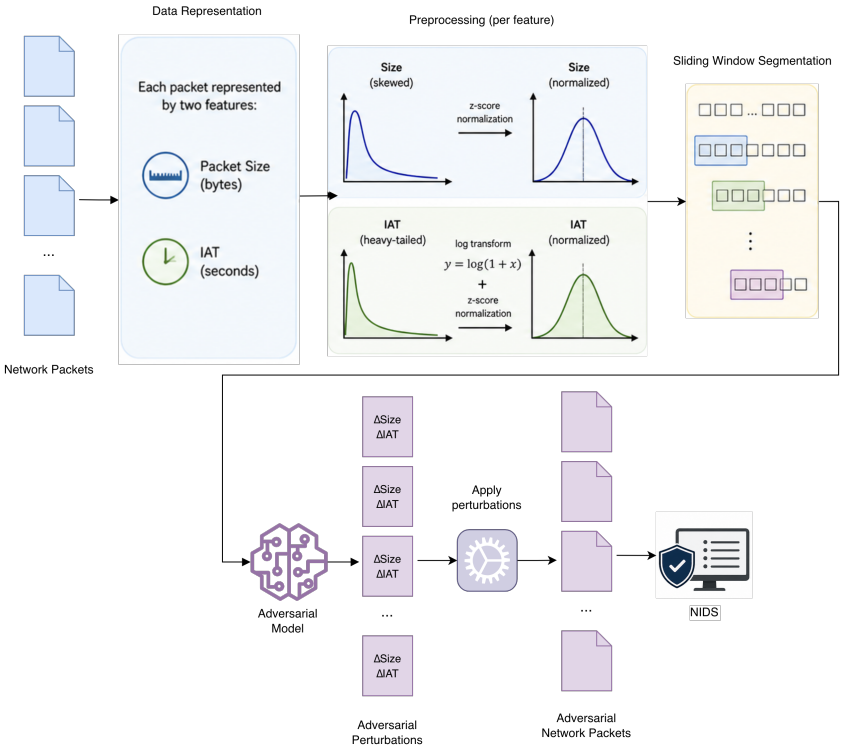


Figure 4.2: Data Representation Overview

4.2.2 Model selection

As mentioned previously, using transformers to generate adversarial perturbations represents a promising approach for exposing vulnerabilities in Network Intrusion Detection Systems (NIDSs). The first step is to select an appropriate architecture for the task.

Unlike traditional sequence generation problems, the objective in this work is not to generate a new sequence from scratch, but rather to produce a sequence of perturbations aligned with the input. Given an input sequence of packets, the model outputs a perturbation vector for each packet, modifying the packet size and IAT features. Therefore, the input and output sequences share the same length and are position-wise aligned. This formulation can be seen as a sequence-to-sequence regression problem with positional alignment, where each output element depends on the full input context but remains strictly tied to its corresponding input position.

Due to this formulation, encoder-only transformer architectures are more suitable than encoder-decoder or decoder-only models. Encoder-decoder models are typically designed for tasks where the output sequence may differ in length or structure from the input (e.g., machine translation), while decoder-only models are commonly used for autoregressive generation. In contrast, encoder-only architectures are well-suited for tasks where each output depends on the full input context but remains aligned with it.

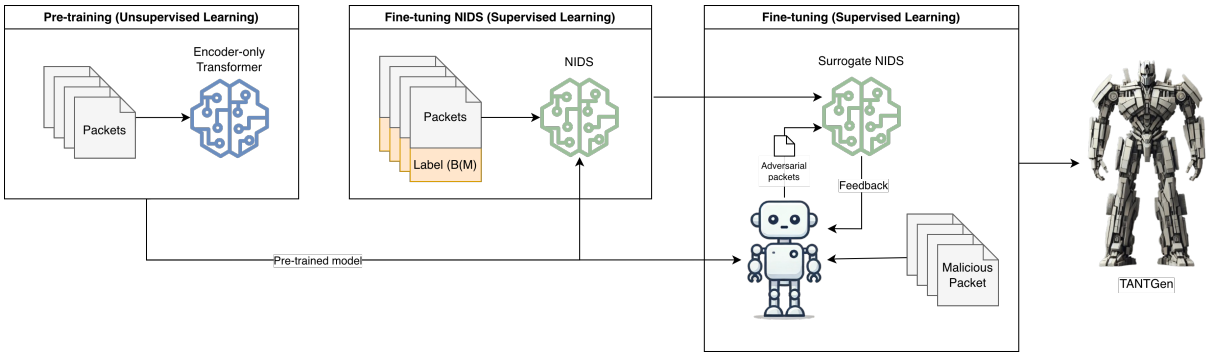


Figure 4.3: TANTGen training process

In this work, we adopt an encoder-only transformer architecture. The model processes the entire input sequence through self-attention layers, capturing both local and global dependencies between packets. This allows each packet representation to incorporate information from the entire flow, which is critical for modeling temporal patterns and crafting coherent adversarial perturbations. The resulting contextual representations are then mapped, through a feed-forward output head, to produce perturbations for each feature of each packet.

This design allows the model to generate context-aware adversarial modifications while maintaining a simple and efficient architecture. Additionally, the encoder structure integrates naturally with the surrogate NIDS, which is used during fine-tuning to guide the generation of perturbations towards successful evasion (see Section 4.2.3).

4.2.3 Transformer Training

The model training process follows a self-supervised approach, combining a pre-training phase on a large dataset to learn general representations with task-specific fine-tuning on a smaller dataset. This methodology has been employed in recent studies [34, 35] to train transformer models for networking tasks, including traffic generation. Figure 4.3 provides an overview of the training pipeline.

In the pre-training phase, the model is trained on unlabeled network traffic to learn general temporal and structural patterns. Given an input sequence of packets, a random subset of packet features is masked, and the model is trained to reconstruct the original packet features at those positions, including both packet size and inter-arrival time. Additionally, small stochastic noise is injected into the input during training to encourage robustness and improve generalization. This is formulated as a sequence-to-sequence regression task with positional alignment, where the model leverages full contextual information to infer the missing values. By learning to reconstruct masked packet-level features from their surrounding context under mild input perturbations, the model captures both short-term dependencies and global traffic patterns, providing a strong initialization for the subsequent adversarial fine-tuning

phase.

In the fine-tuning phase, a surrogate NIDS is first constructed to guide the adversarial training process. Specifically, the pre-trained encoder is fine-tuned as a classification model by freezing the pre-trained parameters and attaching a task-specific classification head that replaces the original pre-trained one. The resulting surrogate NIDS maps input traffic sequences to binary labels (benign or malicious), following a supervised learning paradigm. Once trained, its parameters are frozen during the adversarial generation phase. This surrogate model provides a differentiable approximation of the target detector in a black-box setting, enabling the generator to optimize perturbations that increase the probability of evasion while respecting the imposed regularization terms.

After constructing the surrogate model, the pre-trained transformer is adapted to produce adversarial perturbations under a composite objective. In this stage, the pre-trained parameters are unfrozen, allowing the model to fully adapt to the new task. The training objective is formulated as a regularized adversarial optimization problem, combining (i) an evasion objective that encourages the surrogate NIDS to misclassify the modified traffic, and (ii) a perturbation penalty that constrains the magnitude of the applied modifications.

Formally:

$$\mathcal{L}_{\text{total}}(x, \delta) = \lambda_{\text{e}}\, \mathcal{L}_{\text{evasion}}(\mathcal{R}(x, \delta)) + \lambda_{\text{p}}\, \mathcal{L}_{\text{pert}}(\delta), \tag{4.1}$$

where $\mathcal{R}(x, \delta)$ is a reconstruction operator that applies the perturbation δ to x while enforcing validity constraints (e.g., non-negative inter-arrival times and valid packet sizes).

The loss terms are defined as:

$$\mathcal{L}_{\text{evasion}}(\tilde{x}) = \text{mean}\left(\max(0, f_\psi(\tilde{x}) - m)\right), \tag{4.2}$$

$$\mathcal{L}_{\text{pert}}(\delta) = \text{mean}\left(\delta_{\text{IAT}}\right) + \alpha * \text{mean}\left(\delta^2_{\text{size}}\right). \tag{4.3}$$

Notation

$x = (x_1, \ldots, x_T)$ Original packet sequence.

$\tilde{x} = (\tilde{x}_1, \ldots, \tilde{x}_T)$ Adversarial (perturbed) sequence produced by the model; $\tilde{x} = \mathcal{R}(x, \delta)$

δ Perturbation generated by the model.

$\lambda_{\text{e}}, \lambda_{\text{p}}$ Loss weights.

f_ψ Surrogate model.

m Margin parameter.

α Hyperparameter controlling the trade-off between the magnitude of packet size perturbations and

the remaining components of the overall loss function.

Different penalty functions are used for different features to reflect their distinct roles, characteristics, and scales. In particular, a linear penalty is applied to IAT perturbations, capturing the cumulative cost of added delay, where each increment contributes proportionally to the overall modification. In contrast, a quadratic penalty is used for packet size perturbations, which more strongly penalizes large deviations and thus encourages smaller, less detectable changes, promoting realism in the generated traffic. Several alternatives were explored, including applying the same penalty function to both features, with this asymmetric combination ultimately offering the best trade-off between attack success rate and perturbation realism.

Overall, this combined training strategy enables TANTGen to learn the general statistical properties of network traffic while simultaneously developing the capacity to generate adversarial examples that are realistic and effective in evading intrusion detection systems.

4.2.4 Attack application

After training, the generator is used to produce adversarial traffic directly in the traffic domain. Given an input malicious sequence x, the model outputs a sequence of perturbations δ, which are applied to the packet sequence. Although perturbations are learned in a continuous space, they are later constrained and discretized during reconstruction to ensure that the resulting traffic remains valid and realizable. The packet sequence is then converted back to raw network traffic.

When converting the perturbed packet sequence back to raw network traffic, several protocol-aware constraints are enforced to ensure the validity of the generated traffic. First, a conservative Maximum Transmission Unit (MTU) threshold is applied to all packets, capping the total packet size below the standard 1500-byte Ethernet MTU to prevent fragmentation and avoid triggering size-based anomaly detectors. Second, packets belonging to security-sensitive protocols — such as SSH, TLS, SMB, and RDP — are left unmodified, as altering their payload structure would likely break encryption or authentication handshakes, thereby violating protocol semantics. Protocol identification is performed prior to perturbation to ensure that such constraints are applied consistently. Finally, for protocols carrying structured plaintext payloads, padding is introduced in a protocol-compliant and semantics-aware manner. Rather than blindly appending arbitrary bytes, modifications are constrained to syntactically valid and non-critical regions of each protocol. For example, SSDP messages are padded by inserting optional, well-formed header fields within the HTTP-like header section, ensuring compatibility with existing parsers. In all cases, IP and TCP checksums are recomputed to maintain packet validity.

This process yields a valid PCAP file that preserves the structure and semantics of the original flow while incorporating adversarial modifications. The resulting PCAP can be directly replayed or injected into the network, enabling evaluation against the target NIDS in a fully black-box traffic-space setting.

4.3 Implementation

TANTGen was implemented in `Python` (v3.10.12), following a modular design aligned with the architecture described in the previous sections. The implementation is organized into two main components: (i) Data Pipeline and Traffic Reconstruction and (ii) the Transformer-Based Adversarial Model.

4.3.1 Data Pipeline and Traffic Reconstruction

Traffic data preprocessing and feature extraction were implemented using `Pandas` and `NumPy`, while `Scapy` was used for parsing, manipulating, and reconstructing PCAP files, including the enforcement of protocol-aware constraints during the attack application phase.

To construct the sequential input required by the transformer, traffic flows are segmented into fixed-length windows using a sliding window mechanism with a configurable stride. This process builds upon the windowing strategy of the NTT framework [34]. In particular, the original batching logic — which groups window construction into fixed-size batches to avoid element-wise operations and maintain computational efficiency — was preserved. However, the feature selection, normalization pipeline, and padding strategy were redesigned to accommodate the two-feature packet representation (packet size and IAT) and the specific preprocessing steps described in Section 4.2.1, including logarithmic transformation and z-score standardization.

The resulting implementation allows the generation of (possibly overlapping) windows with configurable length and stride, ensuring compatibility with the temporal structure of network traffic.

4.3.2 Transformer-Based Adversarial Model

The core of TANTGen is an encoder-only transformer model implemented using `PyTorch` and trained with `PyTorch Lightning`. The NTT framework was selected as the starting point for this implementation due to its alignment with several key requirements of this work: it adopts a raw packet-level representation, operates directly on sequential traffic data without handcrafted features, and provides a well-structured encoder architecture suited for capturing temporal dependencies in network flows. However, NTT was designed for passive traffic modeling — specifically, next-packet prediction — and required substantial adaptation to support adversarial perturbation generation.

The following components were reused from the original framework:

- Input projection and embedding layer;
- `DataLoader` construction and configuration handling;
- Optimizer configuration pipeline.

All remaining model logic was redesigned. Most notably:

- **Pre-training objective:** A masked reconstruction strategy was implemented, where a random subset of packet features — including both packet size and IAT — is masked and the model is trained to reconstruct the original values. This differs from the next-packet prediction objective used in NTT. Additionally, positional encoding was explicitly introduced to preserve temporal order within each window.
- **Surrogate model:** The pre-trained encoder is repurposed as a binary traffic classifier by attaching a task-specific classification head, following a supervised fine-tuning paradigm. This component was developed independently.
- **Fine-tuning loop:** The model shifts from a reconstruction objective to adversarial optimization. During the forward pass, it generates perturbations that are applied to the input sequence through a reconstruction operator, which enforces domain validity constraints. A frozen surrogate NIDS is embedded in the training loop to provide the evasion feedback.

Overall, this implementation extends the NTT codebase from a passive traffic modeling tool into an active adversarial generation framework, combining pre-training on unlabeled traffic with surrogate-guided fine-tuning to produce valid and evasive adversarial network flows.

4.4 Summary

This chapter presented TANTGen, a transformer-based framework for generating adversarial network traffic. The data representation was defined at the packet level, encoding each packet as a two-dimensional feature vector of packet size and IAT, segmented into fixed-length windows to capture temporal structure. An encoder-only transformer architecture was adopted, suited for the positionally-aligned sequence-to-sequence nature of the perturbation task. The training methodology combines self-supervised pre-training on unlabeled traffic with surrogate-guided adversarial fine-tuning, enabling the model to learn general traffic representations before adapting to evasion objectives. During attack application, the generated perturbations are projected back into the traffic domain through a reconstruction process that enforces protocol-aware validity constraints, yielding PCAP files that can be directly replayed against a target NIDS. Finally, the implementation was described, detailing the components reused from the NTT framework and the substantial extensions introduced to support the adversarial generation setting.

5

Evaluation

Contents

5.1 Experimental Settings 37
5.2 Results 44
5.3 Summary 57

This chapter presents the evaluation of TANTGen, assessing its ability to generate adversarial network traffic that evades ML-based NIDSs while remaining protocol-compliant and functionally malicious. The chapter is organized into two main sections: the Experimental Settings section, which describes the setup and methodology, followed by the Results section, which analyzes the effectiveness of the generated adversarial traffic across multiple attack scenarios and target systems.

5.1 Experimental Settings

Here, we detail the experimental methodology adopted to evaluate TANTGen. It covers the target NIDSs and feature extractors selected for evaluation, the datasets used, the attacker's capability model, the baselines considered for comparison, the hardware and software setup, and the research questions and metrics used to assess the system.

5.1.1 Target NIDSs and Feature Extractors

In this work, multiple NIDSs were selected as target models to evaluate the effectiveness of the proposed adversarial approach across different detection paradigms.

Firstly, several machine learning-based NIDSs were implemented following a supervised learning-based detection approach. These models include **Logistic Regression (LR)**, **Machine Learning Perceptron (MLP)**, **Random Forest (RF)**, **Support Vector Machine (SVM)**, and **Convolutional Neural Networks (CNN)**. The models are trained using both benign and malicious traffic, enabling them to distinguish between malicious and benign patterns.

For these models, the Python implementation of the **CICFlowMeter** [60, 61] feature extractor was used. CICFlowMeter is a flow-based feature extraction tool that aggregates packets into bidirectional flows based on the standard 5-tuple (source IP, destination IP, source port, destination port, and protocol). It computes a comprehensive set of statistical features, including flow duration, packet counts, byte rates, inter-arrival times, and header-based metrics. This approach provides a higher-level representation of network activity, making it suitable for traditional machine learning classifiers. Notably, this flow-level aggregation transforms packet-level perturbations into aggregate statistical changes, which is key to understanding the effectiveness of different adversarial strategies.

Additionally, **Kitsune** [55] was employed as an anomaly-based NIDS. Kitsune relies on an ensemble of autoencoders to model normal network behavior. Unlike signature-based classifiers, Kitsune is trained exclusively on benign traffic, learning a representation of normal patterns and flagging deviations as anomalies. This characteristic makes Kitsune particularly suitable for evaluating adversarial attacks that aim to mimic legitimate traffic. In our work, we adopt the version of Kitsune proposed by D. Han et al. [1], as it provides faster execution and enables a more direct comparison with the adversarial attack generator introduced in the same work.

Kitsune employs the **AfterImage** [55] feature extractor, a packet-based method that incrementally computes statistical features from network packets over multiple temporal windows. These features capture temporal and spatial correlations between packets without requiring full flow reconstruction, enabling real-time processing with low computational overhead. This packet-level feature extraction makes Kitsune inherently more sensitive to temporal perturbations, particularly inter-arrival time modifications, compared to flow-based approaches.

5.1.2 Datasets

A robust and balanced dataset is essential for training a transformer model capable of generating adversarial network packets. In this work, two publicly available datasets were used: CIC-IDS2017 [62] and the Kitsune [63] datasets.

The **CIC-IDS2017** dataset contains realistic network traffic, including both benign activity and a wide range of attack scenarios. In this work, a subset of benign traffic from CIC-IDS2017 was used during the pre-training phase of the transformer model, allowing it to learn normal traffic patterns. During fine-tuning, attack categories were selected based on sufficient flow volume to support reliable training and evaluation, with a minimum threshold of approximately 10,000 flows. Attacks falling below this threshold, such as Web Attacks (2,180 flows), Botnet (1,966 flows), Infiltration (36 flows), and Heartbleed (11 flows), were excluded. The following attack categories were retained:

- **Brute Force** (13,835 flows): consists of repeated login attempts aimed at gaining unauthorized access by systematically guessing credentials.
- **Distributed Denial-of-Service (DDoS)** (128,027 flows): aims to overwhelm a target system with a large volume of traffic, thereby disrupting normal service availability.
- **Port Scan** (158,930 flows): involves probing a target to identify open ports and available services, often as a reconnaissance step prior to launching further attacks.

The **Kitsune dataset** includes both benign and malicious traffic captured in Internet of Things (IoT) environments. Attack selection followed two criteria: (i) sufficient packet volume, with a minimum threshold of approximately 500,000 packets, and (ii) adequate initial detection rate by the Kitsune NIDS, ensuring that the adversarial setting is meaningful. Attacks with insufficient packet volume, such as Fuzzing (432,783 packets), Video Injection (102,499 packets), SSL Renegotiation (92,652 packets), OS Scan (65,700 packets), and SYN DoS (7,038 packets), were excluded. Similarly, attacks with an initial ASR exceeding 0.5 without any perturbation were excluded, as this indicates that the detector already fails to reliably identify the attack, rendering the adversarial setting meaningless — this affected Active Wiretap, ARP MitM, and Fuzzing. The following attack types were retained:

- **Mirai Botnet** (642,516 packets): malware that infects IoT devices and forms a botnet used to launch large-scale attacks, particularly DDoS attacks.
- **Simple Service Discovery Protocol (SSDP) Flood** (1,439,604 packets): a reflection-based DDoS attack that exploits the Simple Service Discovery Protocol to amplify traffic toward a victim.

The datasets were carefully partitioned to ensure balance, consistency across experiments, and prevent data leakage. Each attack set was divided into four subsets with distinct purposes: (i) the target training set, containing only benign traffic for Kitsune and both benign and malicious traffic for the remaining ML-based NIDSs; (ii) the surrogate model set, containing both benign and malicious traffic, as the surrogate NIDS is trained in a supervised manner and therefore requires labeled examples of both classes to learn a meaningful decision boundary; (iii) the adversarial training set, composed exclusively

of malicious traffic; and (iv) the test set, containing both benign and malicious samples, used to evaluate detection performance and adversarial effectiveness.

For the CIC-IDS2017 dataset, whose target models are flow-based, the split was performed at the flow level to ensure the integrity of flow-based features and maintain temporal consistency. Since the adversarial training set contains only malicious traffic, benign and malicious flows follow different distribution schemes. Benign flows were distributed as 35% for the target training set, 35% for the surrogate set, and 30% for the test set. Malicious flows were distributed as 35% for the target training set, 35% for the surrogate set, 20% for the adversarial training set, and 10% for the test set.

For the Kitsune dataset, the partitioning was performed at the packet level and designed to balance the number of packets across subsets, given the available benign and malicious traffic. This packet-level partitioning aligns with the feature extractor, which operates on streaming packet statistics rather than pre-aggregated flows. Since the target training set contains only benign traffic and adversarial training set contains only malicious traffic, benign and malicious packets follow different distribution schemes, as was the case for the CIC-IDS2017 dataset. For the Mirai Botnet attack (121,621 benign and 642,516 malicious packets), benign packets were distributed as 60% for the target training set, 20% for the surrogate set, and 20% for the test set. Malicious packets were distributed as 10% for the surrogate set, 60% for the adversarial training set, and 30% for the test set. For the SSDP Flood attack (2,637,662 benign and 1,439,604 malicious packets), benign packets were distributed as 80% for the target training set, 10% for the surrogate set, and 10% for the test set. Malicious packets were distributed as 20% for the surrogate set, 60% for the adversarial training set, and 20% for the test set.

Furthermore, for models trained on both benign and malicious traffic, class balancing was applied through random undersampling of the majority class, mitigating bias and improving generalisation performance.

5.1.3 Attacker's Capability

In this work, the attacker is modeled as an end-host adversary, capable of generating and shaping its own malicious traffic (e.g., malware or a controlled client), but without the ability to modify arbitrary in-transit traffic (i.e., no man-in-the-middle capabilities).

As mentioned above, two types of perturbation are considered: modification of packet sizes, achieved by appending payload bytes, and modification of Inter-Arrival Time (IAT), achieved by introducing delays between packets. Both perturbation types are constrained to preserve the functional semantics of the attack traffic and to comply with protocol specifications.

To assess the impact of operational constraints on evasion effectiveness, the following restrictions on attacker capability were considered. Size and IAT restrictions are defined independently and can be combined.

Size restrictions:

- **Unrestricted (100%):** The attacker can append payload to all packets in the flow without restriction on which packets are modified.
- **Restricted (50%):** The attacker can only append payload to up to 50% of the packets in a flow.
- **Restricted (20%):** The attacker can only append payload to up to 20% of the packets in a flow.

IAT restrictions:

- **Unrestricted:** The attacker can delay packet transmission freely, with no bound on the resulting flow duration.
- **Restricted (ratio = x):** The attacker can delay packet transmission such that the resulting flow duration is at most x times the original. Several values of this ratio were explored to identify the minimum perturbation required to successfully evade detection.

These constraints reflect realistic limitations an attacker may face, such as avoiding obvious traffic anomalies, maintaining maliciousness, and reducing the risk of detection by complementary monitoring mechanisms. The combination of size and IAT perturbations is also evaluated to determine whether jointly applying both strategies yields higher evasion rates than either alone.

5.1.4 Baseline

The primary baseline consists of the original, unmodified malicious traffic — that is, the raw attack packets without any adversarial perturbation. This baseline reflects the detection performance of each target NIDS under standard conditions and serves as the reference against which the evasion gains of TANTGen are measured. In this context, the corresponding Attack Success Rate (ASR) represents the fraction of malicious samples that already evade detection without any perturbation, providing a lower bound for adversarial effectiveness.

Additionally, for the Mirai Botnet attack against Kitsune, a comparison is made with TrafficManipulator [1], using the example provided in the publicly available repository, to offer a point of reference against an existing adversarial approach. This comparison is limited to the Mirai Botnet dataset because the adversarial model introduced by D. Han et al. generates adversarial traffic in two phases — adversarial feature generation followed by packet reconstruction — and only the code for the second phase is publicly available, along with a pre-computed feature output file for this specific dataset. Without access to the first-phase code, reproducing the attack pipeline for other datasets was not possible. Therefore, this comparison should be interpreted as a case study rather than a comprehensive benchmark across datasets.

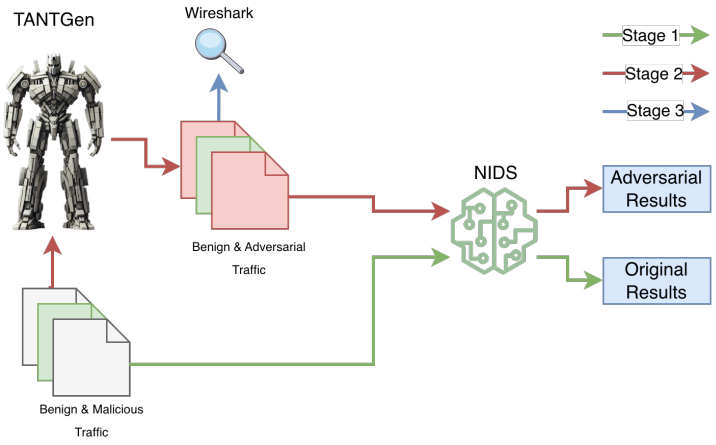


Figure 5.1: TANTGen Evaluation

5.1.5 Setup

The experiments were conducted in a controlled environment designed to simulate realistic network conditions, including a mixture of benign and malicious traffic. The main objective was to evaluate the effectiveness of the proposed adversarial traffic generator in compromising the detection capabilities of ML-based NIDSs.

The experimental setup consists of three main stages. First, baseline experiments are performed. Subsequently, TANTGen is activated, and the evaluation is repeated to measure the degradation in detection performance. When activated, TANTGen processes the traffic by modifying malicious samples while preserving their functionality. Finally, **Wireshark** is used to inspect packet structures and ensure protocol compliance of the generated traffic, as well as to support the validation and analysis of the experimental results. This validation is qualitative and focuses on packet well-formedness and protocol compliance rather than full indistinguishability from legitimate traffic.

Figure 5.1 illustrates this pipeline.

All experiments were conducted on a machine running Ubuntu 22.04.5 LTS with Python 3.10.12. The system was equipped with 192 GB of RAM, two Intel Xeon Gold 5320 processors (52 cores total, clocked at 2.2 GHz), four NVIDIA RTX A4000 GPUs, and two Intel Optane Memory modules of 128 GB each.

Table 5.1 summarises the hyperparameters used across all training stages of TANTGen. All three training phases — pre-training, fine-tuning, and surrogate model training — share the same training hyperparameters, with a maximum of 100 epochs and early stopping applied when convergence is detected. The reported values were selected after experimenting with alternative configurations, and were found to provide a good balance between model capacity and training stability.

Table 5.1: TANTGen Hyperparameters

Hyperparameter	Value
Transformer Architecture	
Number of attention heads	8
Number of layers	6
Linear size	120
Dropout	0.2
Training	
Learning rate	1×10^{-4}
Weight decay	1×10^{-5}
Batch size	64
Maximum epochs	100
Adversarial Generation	
Sliding window size	64

5.1.6 Research Questions and Metrics

Prior to the adversarial evaluation, the detection performance of each target NIDS on the original, unmodified traffic is assessed using the **F1 score**, **Precision**, and **Accuracy**, ensuring that all models are capable of reliably detecting the attacks before any perturbation is applied. This step establishes the validity of the experimental setup and confirms that any observed reduction in detection performance is attributable to the adversarial modifications rather than to weaknesses in the baseline models.

We then aim to address four main research questions to evaluate our solution:

- First, regarding **effectiveness**, we evaluate whether the adversarial samples can evade detection by existing NIDSs, quantified using the **Attack Success Rate (ASR)** — the percentage of adversarial samples that successfully evade detection by the target NIDS, given by:

$$\text{ASR} = \frac{\text{Number of adversarial samples that evade detection}}{\text{Total number of adversarial samples}} \tag{5.1}$$

- Second, regarding **realism**, we assess whether the generated adversarial samples adhere to protocol rules and remain well-formed and protocol-compliant to legitimate network traffic, which is verified qualitatively through protocol compliance inspection using Wireshark.

- Third, regarding **maliciousness**, we evaluate qualitatively whether the generated samples retain their intended malicious functionality after the perturbations are applied, thereby preserving the original attack behaviour.

- Fourth, regarding **operational feasibility**, we assess whether TANTGen can generate adversarial traffic within the timing constraints of a real-time deployment, quantified through **throughput** — the number of packets processed per second — and **latency** — the processing time per packet —

alongside **resource usage**, measuring the average CPU, RAM, and GPU utilization during attack generation.

5.2 Results

In this section, we present the experimental results of TANTGen across the defined research questions. We begin by evaluating the effectiveness of the generated adversarial traffic against ML-based NIDSs on the CIC-IDS2017 dataset, covering the Brute Force, DDoS, and Port Scan attack scenarios. We then assess adversarial performance against Kitsune on the Kitsune dataset, covering the Mirai Botnet and SSDP Flood attacks, including a comparison with TrafficManipulator on the Mirai Botnet case. Finally, we discuss the realism and maliciousness of the generated adversarial samples, as well as the operational feasibility of their application.

5.2.1 Adversarial Results on CIC-IDS2017

Figure 5.2 presents the baseline detection performance of each target ML-based NIDS on the original, unmodified traffic, measured using the F1 score, which provides a balanced view of the model's overall detection capability across both (benign and malicious) classes. All models achieve high F1 scores across DDoS and Port Scan attacks, confirming reliable baseline detection for these attack types. For Brute Force, however, the F1 scores are notably lower — particularly for LR (0.636), MLP (0.699) and SVM (0.728). This is primarily driven by lower precision values (between 0.47 and 0.71 in most of the models), despite high accuracy (around 0.99), indicating a tendency to classify benign traffic as malicious, thus producing a higher number of False Positives. The considerably smaller number of flows available for this attack type (13,835 flows, compared to over 100,000 for the other two categories) likely compounds this effect, limiting the models' ability to learn sufficiently discriminative features. These factors should be taken into account when interpreting the adversarial results for this attack type.

Table 5.2 presents the full adversarial results, including both unrestricted and restricted perturbation scenarios across all three attack types and models. Here, Attack Success Rate (ASR) is used as the evaluation metric, as it directly captures the attacker's ability to evade detection regardless of the detector's behaviour on benign traffic. It should be noted that IAT-only restricted scenarios are not included in the table, as the model was found to apply negligible IAT perturbations on this dataset, a behaviour that will be analysed further in this section.

For a clearer overview of the maximum evasion potential of each perturbation type, Figure 5.3 illustrates only the unrestricted results (IAT, Size 100%, and Both 100% columns), where both IAT delays and payload padding can be applied freely without any constraints. Brute Force was the attack type for which evasion was least consistent across models, with several models failing to reach high ASR values

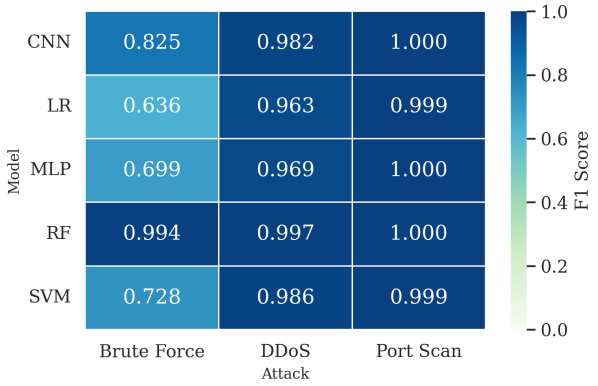


Figure 5.2: Baseline Detection Performance on CIC-IDS2017

Table 5.2: Attack Success Rate results on CIC-IDS Dataset

		Feature	Original	IAT	Size			Both		
		Pkt. limit [a]			100%	50%	20%	100%	50%	20%
Attack	**Model**									
Brute Force	CNN		0.025	0.012	0.348	0.050	0.228	0.348	0.025	0.025
	LR		0.037	0.400	1.000	0.400	0.522	1.000	0.537	0.400
	MLP		0.000	0.037	1.000	0.325	0.337	1.000	0.050	0.050
	RF		0.000	0.037	0.033	0.050	0.043	0.033	0.050	0.050
	SVM		0.012	0.400	1.000	0.650	0.554	1.000	0.425	0.400
DDoS	CNN		0.030	0.032	0.141	0.034	0.032	0.807	0.035	0.029
	LR		0.047	0.050	0.360	0.168	0.045	1.000	0.045	0.042
	MLP		0.055	0.059	0.360	0.174	0.055	1.000	0.052	0.051
	RF		0.006	0.006	1.000	0.997	0.884	0.999	0.999	0.879
	SVM		0.011	0.014	0.356	0.134	0.010	1.000	0.005	0.004
Port Scan	CNN		0.001	0.275	0.997	0.826	0.374	1.000	0.875	0.543
	LR		0.000	0.000	1.000	0.175	0.005	1.000	0.170	0.003
	MLP		0.000	0.004	1.000	0.815	0.376	1.000	0.816	0.376
	RF		0.000	0.002	1.000	0.831	0.376	1.000	0.830	0.375
	SVM		0.000	0.000	0.937	0.515	0.349	1.000	0.798	0.373

Colour scale (ASR): Below baseline | ≈ baseline | 0.1 – 0.3 | 0.3 – 0.6 | 0.6 – 0.95 | ≥ 0.95

[a] Percentage of packets eligible for size modification

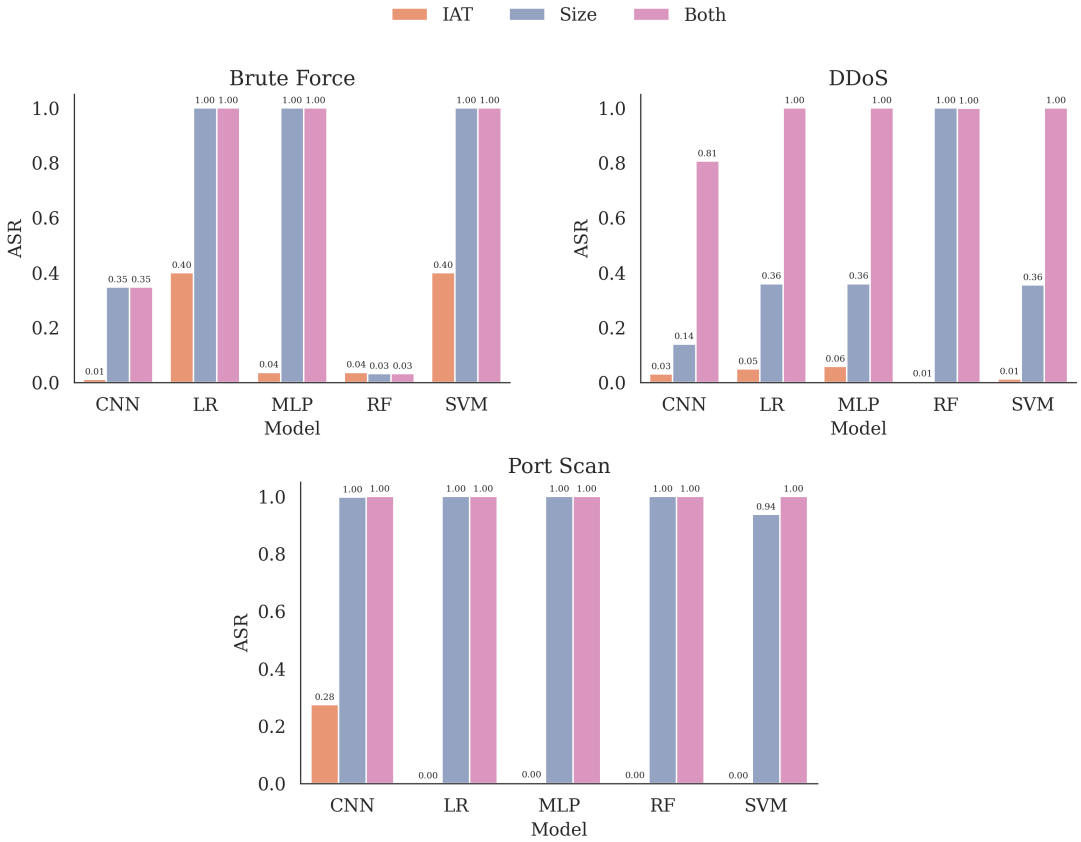


Figure 5.3: Attack Success Rate on CIC-IDS dataset

even in the unrestricted setting. This may be partly explained by the characteristics of the dataset itself, as we noticed that the surrogate model also struggled to achieve its best performance on this attack type, potentially limiting the quality of the adversarial perturbations generated. The following analysis examines the results in detail, discussing the impact of each perturbation type and the effect of the imposed constraints on evasion effectiveness.

A – Impact of size perturbation. Size perturbation, considering packet size modifications only and no IAT changes, consistently proves to be the most effective strategy across all attack types and models in the CIC-IDS setting. In the size unrestricted setting (Size 100%), it achieves an ASR of 1.000 against most models for Port Scan, and yields strong results for Brute Force and DDoS as well. The combined perturbation of both size and IAT generally matches or slightly improves upon size-only results in the unrestricted setting, suggesting that the dominant contribution comes from size modifications. This indicates that flow-based features extracted by CICFlowMeter are highly sensitive to byte-count statistics, which are directly impacted by size manipulation. Port Scan stands out as the attack most susceptible to adversarial manipulation, with near-perfect ASR achieved across all models under the unrestricted size

perturbation.

B – Impact of IAT perturbation. Although IAT perturbation alone produces modest evasion gains in the unrestricted setting—with the adversarial model applying only minimal modifications corresponding to a ratio between $1.002\times$ and $1.01\times$—this behaviour is an expected consequence of the black-box threat model adopted. These results highlight a potential limitation of the surrogate modeling approach: since TANTGen optimizes perturbations against a packet-level surrogate rather than the flow-level target models directly, the surrogate may fail to accurately capture how local timing perturbations accumulate into global flow statistics. As a result, the model learns to minimize IAT perturbations, as small timing changes appear sufficient to fool the surrogate while keeping the perturbation penalty low, leading to suboptimal exploration of the IAT perturbation space — even if the target models may be vulnerable to larger IAT modifications.

To investigate this limitation, Figure 5.4 presents results obtained with a modified version of TANTGen that explicitly constrains the IAT perturbation to approach a given ratio, allowing the study of the evasion potential of IAT modifications beyond what the model spontaneously applies. These results confirm that forcing a higher IAT budget does improve ASR. The effectiveness varies considerably across attack types: for Brute Force, high ASR is achieved with a relatively small ratio of approximately $1.25\times$, suggesting that the target models are sensitive to even modest timing modifications for this attack. For DDoS, the improvement is more gradual, requiring substantially higher ratios to achieve meaningful evasion gains. Port Scan represents an intermediate case, where ASR improves with increasing ratio but plateaus at around 0.5 or below for most models.

C – Impact of combined perturbation. Combining size and IAT perturbations does not always yield better results than applying size modifications alone, as can be observed in Table 5.2, particularly under restricted settings where only a subset of packets can be modified. This may seem counterintuitive, but it is again a consequence of the surrogate-based black-box optimization: when both perturbation types are applied simultaneously, the model distributes the perturbation budget across two strategies, which may result in insufficient packet padding to achieve the same evasion effectiveness as when focusing exclusively on size modifications. In other words, the joint optimization leads the model to a trade-off solution that fools the surrogate but is not necessarily optimal against the flow-level target models, further illustrating the inherent challenge of black-box adversarial attacks when the surrogate and target models operate at different levels of abstraction.

D – Effect of size perturbation constraints. Figure 5.5 illustrates how the ASR degrades as the proportion of packets eligible for size perturbation is reduced from 100% to 50% and 20%, with only

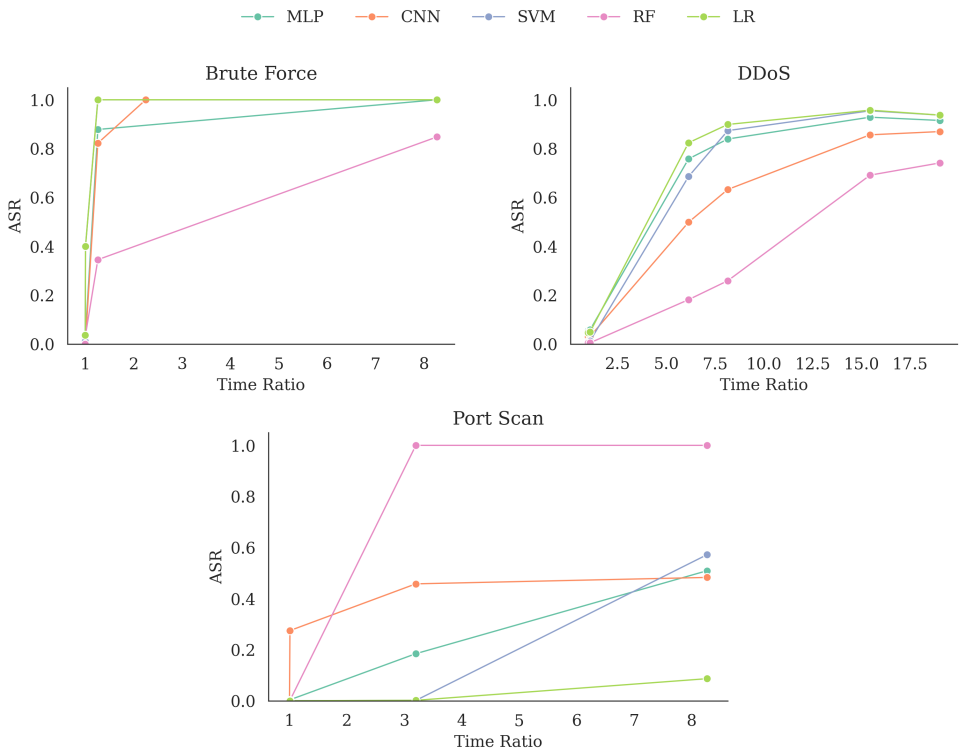


Figure 5.4: Difference of Attack Success Rate if force IAT perturbation (only IAT was perturbed here)

size perturbation applied. The impact of these constraints varies considerably across attack types and models, revealing differences in how models rely on flow-level statistics.

For **Brute Force**, the behaviour is notably non-monotonic for several models. Most models show an increase in ASR at 20%, followed by a partial decrease at 50%, before recovering or improving at 100%. This irregular pattern may reflect the instability of the adversarial perturbations on this attack type, consistent with the already observed difficulty of TANTGen in achieving reliable evasion for Brute Force. RF remains consistently near zero across all constraint levels, confirming its robustness against size perturbation for this attack type.

For **DDoS**, the behaviour is dominated by RF, which rises sharply to near-perfect ASR already at 20% and maintains it throughout, in stark contrast to all other models, which remain at very low ASR values across all constraint levels. This indicates that RF heavily relies on size-dependent aggregate features, which can be significantly altered even with sparse perturbations, whereas the remaining models appear to depend on more robust or diverse feature combinations.

For **Port Scan**, the ASR increases steadily and consistently across all models as more packets are perturbed, reaching near-perfect evasion at 100%. Even at 20%, the ASR is already substantial for most models, suggesting that modifying a relatively small proportion of packets is sufficient to shift the flow-level statistics enough to evade detection. This confirms that Port Scan detection is highly dependent

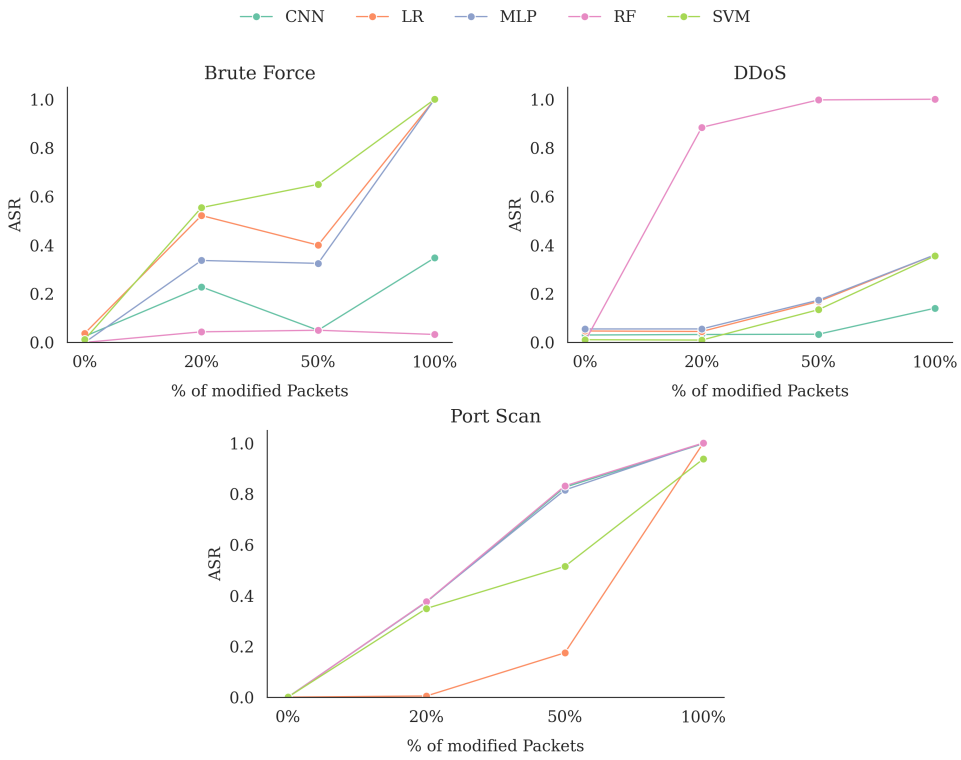


Figure 5.5: Difference of Attack Success Rate if limit size perturbation (only size was perturbed here)

on aggregate statistical patterns rather than fine-grained structure. LR is the slowest to be fooled, only reaching high ASR values at 100%.

Overall, these results suggest that adversarial effectiveness in this dataset is primarily driven by the mismatch between packet-level perturbations and flow-level feature aggregation. Attacks that directly influence aggregate statistics (e.g., size) are highly effective, whereas those that require consistent temporal manipulation (e.g., IAT) are harder to exploit under a packet-level surrogate model. This highlights the importance of aligning the adversarial generation process with the feature representation used by the target NIDS.

5.2.2 Adversarial Results on Kitsune

Figure 5.6 presents the baseline detection performance of Kitsune on the original, unmodified traffic, measured using the F1 score. Kitsune achieves high F1 scores for both attack types — 0.939 for Mirai Botnet and 0.998 for SSDP Flood — confirming that it reliably detects both attacks before any perturbation is applied. This establishes a strong baseline, ensuring that any degradation in performance can be attributed to the adversarial perturbations rather than inherent model weaknesses.

Tables 5.3 and 5.4 present the full adversarial results against Kitsune for the Mirai Botnet and SSDP Flood attacks, respectively, including both size and IAT perturbation scenarios across multiple ratio val-

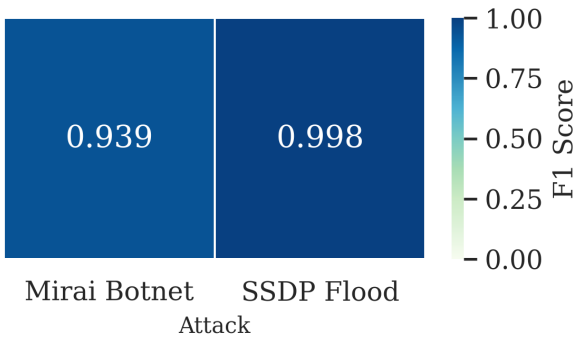


Figure 5.6: Baseline Detection Performance on Kitsune dataset

ues. Unlike the CIC-IDS setting, size restrictions on the percentage of packets eligible for modification are not included, as preliminary experiments showed that restricting the number of modified packets had no impact on the results. For IAT perturbation, the last row of each table corresponds to the ratio spontaneously applied by the unmodified version of TANTGen ($8.38\times$ for Mirai and $63.6\times$ for SSDP Flood), while the remaining rows were obtained using the modified version of TANTGen that explicitly constrains the IAT perturbation to approach a given ratio, allowing the study of the minimum perturbation required to achieve successful evasion.

Table 5.3: Mirai Botnet's ASR results against Kitsune

Feature	Time Ratio	Attack: Mirai Botnet
Original		0.116
IAT	5.48	0.133
	6.4	0.160
	7.25	0.560
	7.59	1.000
	8.38	1.000
Size		0.116
Both		0.116

Table 5.4: SSDP Flood's ASR results against Kitsune

Feature	Time Ratio	Attack: SSDP Flood
Original		0.000
IAT	3.82	0.002
	20.61	0.185
	32.43	0.419
	51.3	1.000
	63.6	1.000
Size		0.002
Both		0.002

Colour scale (ASR): Below baseline | $\approx$ baseline | 0.1 – 0.3 | 0.3 – 0.6 | 0.6 – 0.95 | $\geq$ 0.95

Figure 5.7 complements the table by illustrating the ASR achieved by each perturbation type in the original TANTGen and unrestricted setting for both attacks. In contrast to the CIC-IDS2017 results, where size perturbation dominated, Kitsune exhibits a markedly different behaviour, as discussed below. The following analysis discusses the impact of each perturbation type, the behaviour of combined perturbations, the ratio thresholds required to achieve successful evasion, and a comparison with TrafficManipulator.

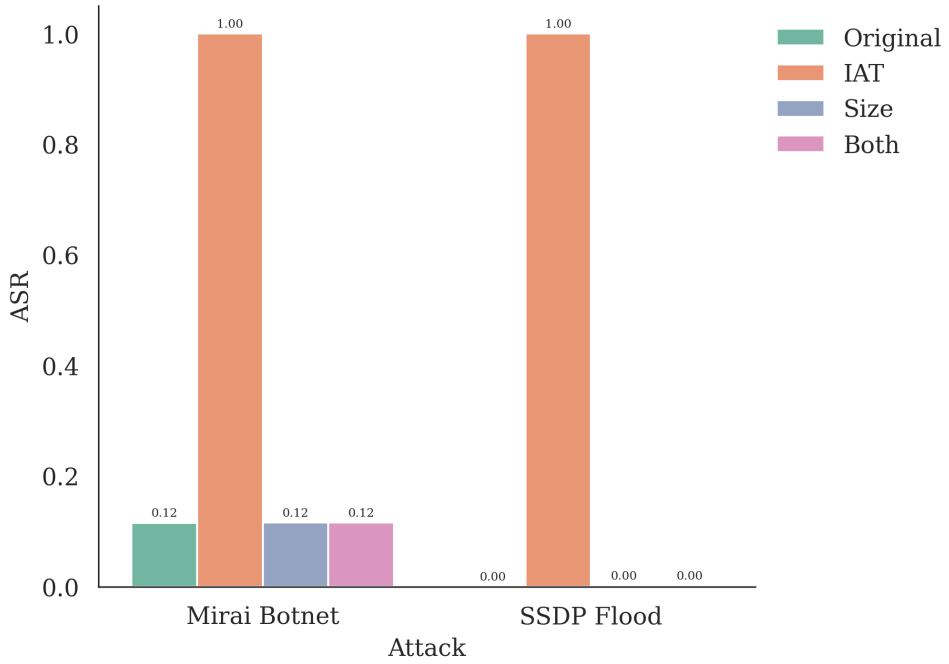


Figure 5.7: Attack Success Rate on Kitsune dataset

A – Impact of size perturbation. Unlike the results observed on the CIC-IDS2017 dataset, size perturbation has virtually no effect on Kitsune's detection rate for either attack, with ASR values remaining near the baseline results regardless of the perturbation applied. To confirm this finding, additional arbitrary size perturbation strategies were tested, none of which yielded meaningful evasion gains—and in some cases even degraded performance.

This result highlights a fundamental difference between flow-based and packet-based detection: while CICFlowMeter-derived models rely heavily on aggregate size statistics, Kitsune's AfterImage representation captures fine-grained temporal dynamics, making it largely insensitive to payload size manipulations.

B – Impact of IAT perturbation. In contrast, IAT perturbation proves to be the decisive factor for evading Kitsune. As shown in Figure 5.8, the ASR increases sharply once the ratio exceeds a certain threshold, reaching a perfect evasion rate of 1.000 for both attacks. For Mirai Botnet, full evasion is achieved at a ratio of $7.59\times$. For SSDP Flood, however, a substantially higher ratio of $51.3\times$ is required. It is worth noting that the two attacks exhibit fundamentally different temporal characteristics: the Mirai Botnet traffic spans 779.28 minutes over 192,755 packets, corresponding to an average of approximately 247 packets per minute, whereas the SSDP Flood concentrates 287,921 packets into only 11.23 minutes, yielding an average of approximately 25,640 packets per minute — roughly 104 times more packets per minute than Mirai. This results in very different average inter-arrival times between the

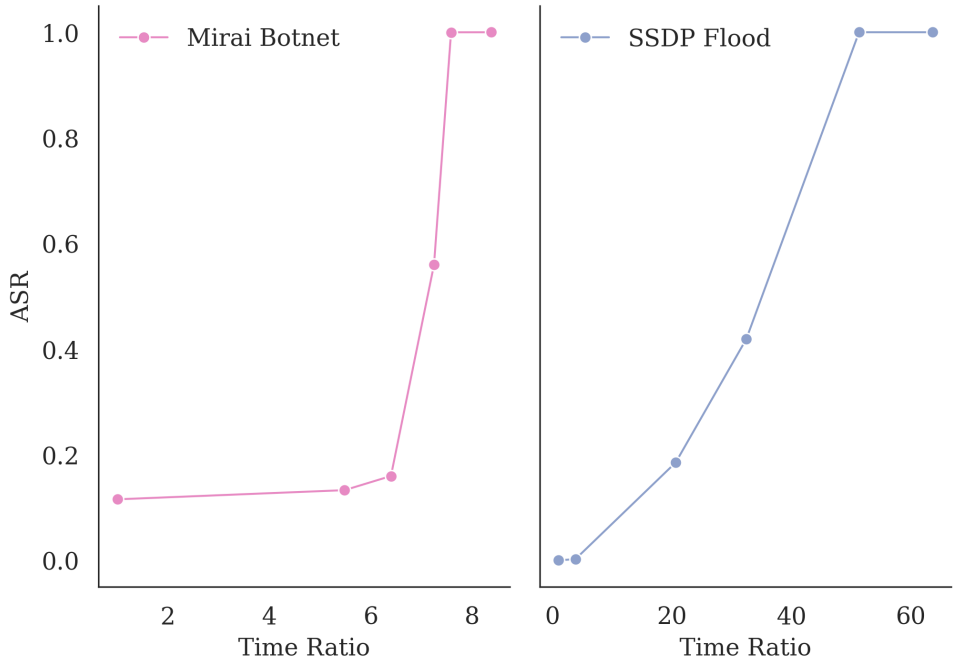


Figure 5.8: Difference of Attack Success Rate if limit IAT perturbation (perturbation only in size)

two attacks, which may help explain the large difference in the ratio required for successful evasion. In particular, for SSDP Flood, even large multiplicative ratios translate into relatively small absolute timing shifts, requiring a more aggressive perturbation rate to significantly alter the statistical features captured by Kitsune.

C – Impact of combined perturbation. As observed in Tables 5.3 and 5.4, combining size and IAT perturbations yields no improvement over either strategy applied in isolation— in fact, the combined ASR remains at or near the baseline for both attacks. This result is particularly striking given that IAT perturbation alone achieves perfect evasion at sufficiently high ratios. The explanation mirrors the one discussed for the CIC-IDS setting: when both perturbation types are optimized simultaneously in a black-box setting, the model is forced to balance two competing objectives against a surrogate that may not accurately reflect the target's sensitivity to each feature. In the Kitsune setting, where evasion is driven almost exclusively by IAT modifications, this trade-off is especially costly—the model allocates part of the perturbation budget to size modifications that have a negligible effect on Kitsune, at the expense of the IAT perturbations that actually matter, resulting in a combined solution that fails to achieve meaningful evasion.

Overall, these results indicate that Kitsune's detection mechanism is more sensitive to temporal structure, and that successful evasion requires shifting the traffic distribution outside the learned temporal profile of benign behaviour.

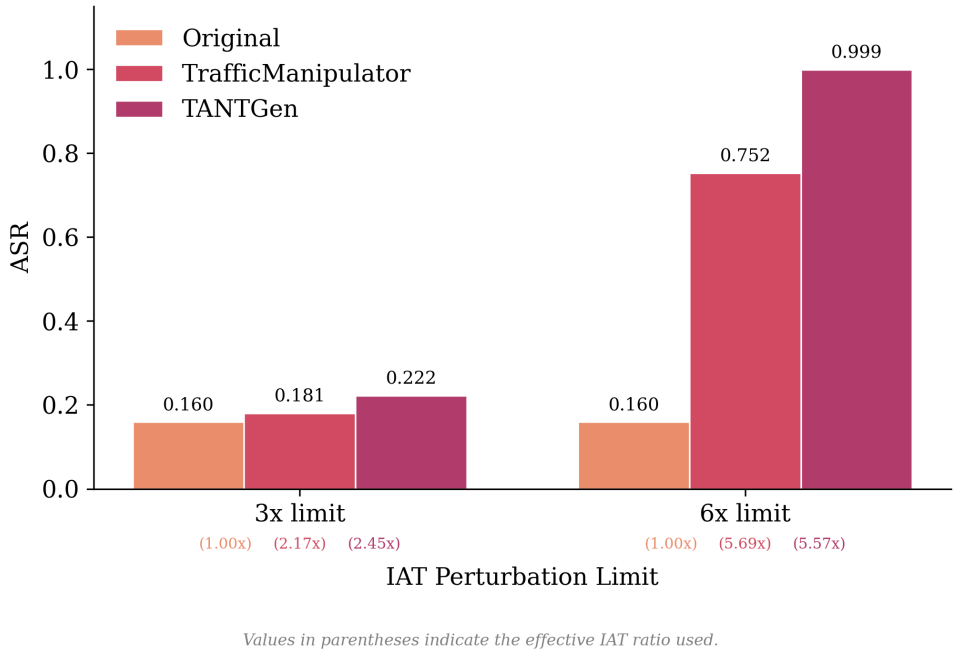


Figure 5.9: TANTGen vs TrafficManipulator — Mirai Botnet (Kitsune)

D – Comparison with TrafficManipulator. TANTGen is additionally compared against TrafficManipulator [1] on the Mirai Botnet example provided in its public repository, consisting of 70,000 benign packets for training Kitsune and 10,000 malicious packets for evaluation. Two time ratio constraints are considered: $3\times$ and $6\times$ the original inter-arrival time (corresponding to additive increases of $2\times$ and $5\times$ as reported in [1]). Figure 5.9 presents the results of the baseline attack, as well as TrafficManipulator and TANTGen performances. As shown, the baseline ASR on unmodified traffic is 0.160, reflecting that Kitsune already fails to detect a portion of the original Mirai packets.

Under the $6\times$ constraint, TrafficManipulator achieves an ASR of 0.752, while TANTGen reaches 0.999, using an effective ratio of $5.57\times$—remaining within the imposed limit. Under the stricter $3\times$ constraint, TrafficManipulator achieves an ASR of 0.181, while TANTGen achieves 0.222 at an effective ratio of $2.45\times$. In both cases, TANTGen outperforms TrafficManipulator while operating within the same constraints.

These results demonstrate that TANTGen is able to exploit temporal vulnerabilities more efficiently, achieving higher evasion rates under the same or tighter constraints. This advantage likely stems from its end-to-end learning formulation, which directly operates in the traffic domain, whereas TrafficManipulator relies on a two-stage pipeline with feature-level generation followed by heuristic packet reconstruction.

Taken together, the results on CIC-IDS2017 and Kitsune demonstrate that the effectiveness of adversarial perturbations is tightly coupled to the feature representation used by the target NIDS: flow-based models are highly vulnerable to size manipulations, whereas packet-based models relying on temporal

statistics are primarily vulnerable to timing perturbations.

5.2.3 Realism and Maliciousness

A – Realism. Protocol compliance of the generated adversarial traffic was verified using Wireshark. It demonstrated that the modified packets were well-formed and compliant with their respective protocol specifications. Packet sizes remained within the conservative MTU threshold, avoiding fragmentation. No protocol violations were observed, supporting the conclusion that the adversarial modifications preserve syntactic validity and protocol compliance at the packet level.

However, this validation does not fully capture higher-level notions of realism. More comprehensive assessments could include statistical comparisons with benign traffic (e.g., distributions of packet sizes and inter-arrival times) or replay-based validation to confirm correct end-to-end protocol behaviour. Such analyses are left for future work due to time constraints.

B – Maliciousness. The impact of adversarial perturbations on the malicious functionality of each attack type is assessed qualitatively.

For **Brute Force** attacks, the perturbation poses virtually no risk to maliciousness. All login attempts are still transmitted, and the only effect of IAT perturbation is a slower submission rate, which does not prevent the attack from achieving its objective.

For **DDoS** attacks, the impact depends on the perturbation type applied. Size perturbation increases packet sizes, which may in fact amplify the volumetric impact of the attack. However, IAT perturbation reduces the packet transmission rate, which could degrade the flooding intensity. While the applied ratios are modest and the traffic volume likely remains well above normal thresholds, a reduction in attack effectiveness cannot be ruled out.

For **Port Scan** attacks, size perturbation appends a dummy payload to probe packets, which does not interfere with the scanning mechanism itself. The target ports are still reached and their responses observed, preserving the reconnaissance capability of the attack.

For **Mirai Botnet** traffic, the reasoning is analogous to the DDoS case, as Mirai primarily acts as a vehicle for large-scale flooding attacks. The IAT perturbation required to evade Kitsune ($7.59\times$) reduces the packet rate, which may partially degrade the flood intensity. However, the overall attack impact is still expected to remain significant.

For **SSDP Flood** attacks, this concern is most pronounced. Achieving full evasion requires a ratio of $51.3\times$, which corresponds to a substantial reduction in the packet transmission rate. Given that the effectiveness of a reflection-based flood relies directly on the volume of amplified traffic directed at the victim, such a reduction may meaningfully compromise the attack's impact. This represents the most significant trade-off between evasion and maliciousness observed in this work, highlighting a limitation

of timing-based perturbations for high-rate, short-duration flooding attacks.

5.2.4 Operational Feasibility

Table 5.5 summarizes the operational feasibility of TANTGen across all attack types and perturbation configurations. Results are averaged over all evaluated batch sizes, as batch size was found to have a negligible impact on end-to-end throughput and latency. This observation indicates that the primary bottleneck lies in the pre- and post-processing stages, rather than in the transformer inference itself, highlighting that the current system is predominantly I/O- and CPU-bound rather than model-bound.

Table 5.5: End-to-End Computational Cost of TANTGen Across Attack Types and Perturbation Strategies

Attack	**Feature**	**Throughput** *(pkt/s)*	**Latency** *(ms/pkt)*	**CPU** *(%)*	**RAM** *(%)*	**GPU** *(%)*
Brute Force	IAT	1119.0	0.89	288.4	0.57	5.2
	Size	879.9	1.14	304.5	0.59	5.2
	Both	877.6	1.14	308.6	0.59	4.8
DDoS	IAT	1663.2	0.60	188.0	0.57	7.4
	Size	749.6	1.33	196.3	0.60	7.2
	Both	749.0	1.34	186.6	0.60	7.6
Port Scan	IAT	1912.2	0.52	133.5	0.57	10.7
	Size	748.6	1.34	145.9	0.60	8.7
	Both	743.4	1.35	174.5	0.60	9.0
Mirai Botnet	IAT	2038.9	0.49	107.5	0.58	50.2
	Size	1856.6	0.54	107.4	0.61	27.5
	Both	1863.2	0.54	107.3	0.61	38.1
SSDP Flood	IAT	1820.2	0.55	114.4	0.56	59.3
	Size	915.4	1.09	103.2	0.66	47.3
	Both	918.0	1.09	102.9	0.67	48.6

A – Time breakdown. Figure 5.10 presents the relative contribution of each processing stage to the overall execution time. Packet rebuilding consistently dominates the computational cost across all attack scenarios, accounting for the largest share of total execution time, followed by feature extraction. In contrast, transformer inference contributes only a negligible fraction.

These results indicate that the computational overhead of TANTGen is primarily driven by packet-level operations rather than by the neural model itself. Consequently, optimization efforts should focus on improving the efficiency of the feature extraction and packet reconstruction pipelines, rather than further optimizing the transformer architecture.

B – Throughput and latency. As illustrated in Figure 5.11, TANTGen achieves throughput values ranging from approximately 743 to 2,039 packets per second, depending on the attack type and pertur-

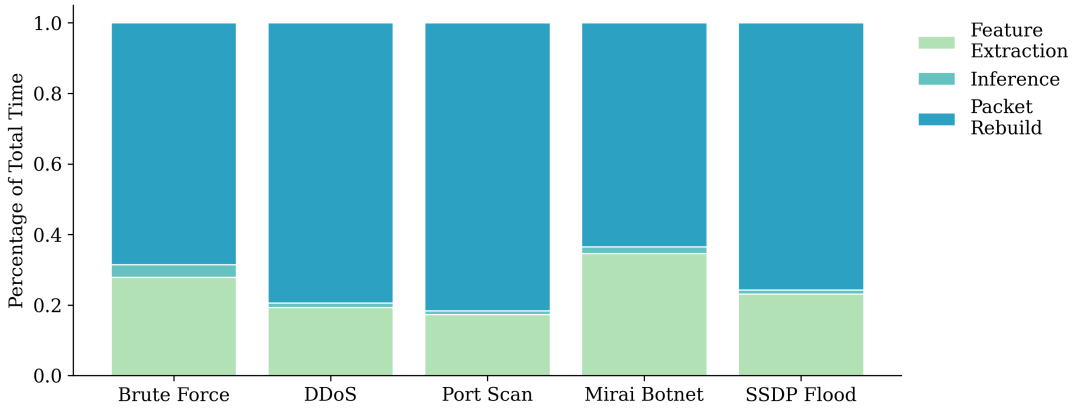


Figure 5.10: Relative contribution of each processing stage to total execution time

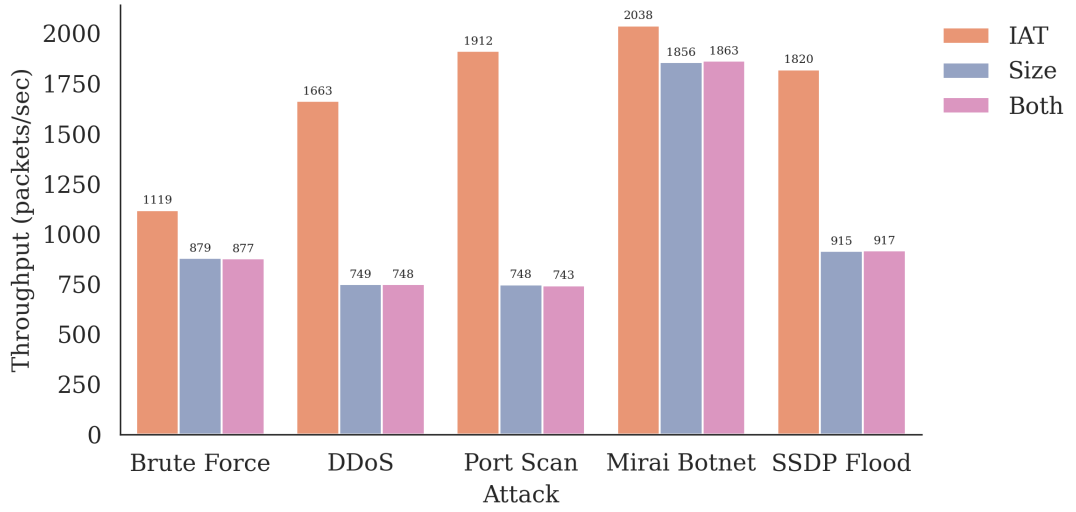


Figure 5.11: Throughput across attack types and perturbation strategies

bation strategy. This corresponds to per-packet latencies between 0.49 and 1.35 ms.

IAT-only perturbation consistently achieves the highest throughput. In contrast, size perturbation introduces additional overhead due to dummy payload insertion and the need to recompute packet fields such as checksums. The combined perturbation strategy yields throughput and latency values very close to those of size-only perturbation, indicating that the additional cost of applying IAT modifications on top of size perturbation is negligible.

The achieved throughput remains significantly below the packet rates typical of high-speed network links. For instance, a 1 Gbps link carrying maximum-size Ethernet frames corresponds to approximately 81,000 packets per second. This gap indicates that TANTGen, in its current form, is not suitable for inline real-time deployment in high-throughput environments. However, the achieved performance may still be sufficient for offline attack generation, replay-based evaluation, or lower-rate attack scenarios. For future

work, other techniques and libraries (e.g., Intel DPDK or other kernel bypass options) can be explored in order to try to make I/O operations more efficient.

C – Resource usage. RAM utilization remains consistently low (below 1%) across all configurations, indicating minimal memory overhead. In contrast, CPU usage is significantly higher, frequently exceeding 100% and in some cases reaching several hundred percent. This reflects effective multi-core utilization, driven primarily by CPU-bound operations in the feature extraction and packet rebuilding stages.

GPU utilization is comparatively modest, ranging from approximately 5% for CIC-IDS2017 attacks to around 50–60% for Kitsune attacks. The imbalance between CPU and GPU usage is expected, as the most computationally intensive stages — feature extraction and packet reconstruction — do not leverage GPU acceleration. As a result, the GPU is primarily engaged during transformer inference. The higher GPU utilization observed in Kitsune scenarios is consistent with the larger packet volumes processed in those experiments, further reinforcing that overall performance is limited by CPU-side processing rather than GPU-based model execution.

5.3 Summary

Overall, the evaluation demonstrates that TANTGen is effective at generating adversarial traffic capable of evading diverse ML-based NIDSs, while preserving protocol compliance and, in most cases, guaranteeing the preservation of malicious functionality. The results highlight that the success of adversarial perturbations is strongly dependent on the feature representation of the target system, with flow-based models being more sensitive to size manipulations and packet-based models to temporal perturbations. At the same time, the analysis reveals an inherent trade-off between evasion and attack strength, particularly for timing-based modifications in high-rate attacks. Finally, the feasibility study shows that the main performance bottleneck of the current system lies in packet-level processing rather than in the transformer model itself, indicating that future optimizations should focus on improving I/O and reconstruction efficiency to enable deployment in higher-throughput environments.

6

Conclusion

Contents

6.1 Main Findings 59
6.2 System Limitations and Future Work 60

This work presented TANTGen, a transformer-based adversarial traffic generator designed to evaluate the robustness of ML-based NIDSs against evasion attacks. By operating directly in the traffic domain and generating perturbations that are both protocol-compliant and functionality-preserving, TANTGen bridges the gap between traffic-space adversarial attacks and practical, deployable evasion strategies.

6.1 Main Findings

The experimental evaluation demonstrated that TANTGen is capable of significantly degrading the detection performance of a diverse set of ML-based NIDSs across multiple attack scenarios and datasets. On the CIC-IDS2017 dataset, size perturbation proved to be the dominant evasion strategy, achieving perfect or near-perfect ASR against most models in the unrestricted setting (IAT perturbation is not limited and all packets are eligible for size modification) for Port Scan, Brute Force, and DDoS attacks. IAT perturbation, by contrast, had a negligible effect on flow-based NIDSs, as the induced timing modifica-

tions were insufficient to meaningfully shift the statistical features extracted by CICFlowMeter. On the Kitsune dataset, the situation was reversed: size perturbation had virtually no impact, while IAT perturbation proved decisive, successfully evading Kitsune for both the Mirai Botnet and SSDP Flood attacks once a sufficient ratio was applied. This complementary behaviour across datasets highlights how different NIDSs rely on fundamentally different feature spaces, and how an effective adversarial approach must adapt its perturbation strategy accordingly.

Beyond evasion performance, the analysis of which perturbation types succeed or fail against each NIDS provides insights into the relative importance of different traffic features for detection. In particular, the results suggest that flow-based NIDSs are more sensitive to size-related features, while packet-based systems such as Kitsune are more reliant on temporal patterns. This distinction is critical, as it reveals that robustness cannot be achieved by hardening a single feature space, but instead requires defenses that account for multiple, complementary representations of network traffic. These findings expose inherent weaknesses in current detection approaches and motivate the study of more robust NIDS architectures — for instance, through adversarial training using generated evasion traffic, or through the design of detectors that jointly exploit multiple, complementary feature spaces to reduce the attack surface available to an adversary.

The comparison with TrafficManipulator on the Mirai Botnet case demonstrated that TANTGen achieved superior evasion rates under the same IAT constraints. Protocol compliance of all generated traffic was confirmed through Wireshark inspection, and a qualitative analysis of maliciousness preservation indicated that the risk to attack functionality is negligible for Brute Force and Port Scan, low-to-moderate for DDoS and Mirai Botnet, and most significant for SSDP Flood, where the high IAT ratio required for evasion may meaningfully reduce the flood intensity. These results highlight an important trade-off between evasion and attack effectiveness, which depends on the nature of the attack and the perturbation required.

Taken together, these results suggest that current ML-based NIDSs remain vulnerable to relatively simple perturbations applied directly in the traffic domain. More broadly, they demonstrate that generating realistic, protocol-compliant adversarial traffic is not only feasible, but also highly effective in exposing systematic weaknesses in modern ML-based network defenses.

6.2 System Limitations and Future Work

Despite the promising results, TANTGen has several limitations that motivate future research directions.

6.2.1 Maliciousness Verification

The preservation of malicious functionality was assessed qualitatively in this work. Future work could include quantitative and experimental validation, for instance by replaying the generated adversarial traffic in a controlled network environment and confirming that the intended attack effects are still observed. This is particularly important for high-rate flooding attacks such as SSDP Flood, where timing perturbations may directly impact attack effectiveness.

6.2.2 Feature Space Expansion

In its current form, TANTGen explores two perturbation dimensions: packet size and inter-arrival time. A natural and important extension would be to incorporate additional traffic features into the adversarial generation process, such as TCP flag sequences, TTL field manipulation, packet reordering, crafted packet injection, or IP header fields. Expanding the perturbation space would likely increase evasion effectiveness against a broader range of NIDSs, while also enabling a more comprehensive analysis of which feature dimensions are most critical for detection. Moreover, exploring interactions between multiple feature dimensions could reveal higher-order attack strategies that remain unexplored in current systems.

6.2.3 Adversarial Robustness of NIDS

TANTGen can naturally serve not only as an attack tool but also as a means to improve the robustness of ML-based NIDSs. By incorporating adversarially generated traffic into the training pipeline—a process known as adversarial training—detection models can be exposed to evasion examples during learning, potentially improving their resilience to such perturbations in deployment. Beyond adversarial training, TANTGen could also be used to benchmark and systematically compare the robustness of different NIDS architectures, closing the loop between attack generation and defense and representing a promising direction for building more robust intrusion detection systems.

6.2.4 Real-Time Deployment

TANTGen was evaluated in a controlled offline setting, processing pre-captured PCAP files rather than live network traffic. While the model itself exhibits low inference latency and is capable of processing packets efficiently, the overall throughput was primarily constrained by the overhead associated with packet parsing, reconstruction, and I/O operations. In particular, the use of Scapy for packet dissection and reassembly introduced significant overhead, especially when processing large PCAP files sequentially. Since the evaluation pipeline operated in a single-threaded manner without parallel processing,

the observed end-to-end performance reflects the limitations of the experimental setup rather than the computational efficiency of the model itself. Alternative high-performance packet processing libraries, such as Intel DPDK, could potentially reduce I/O overhead and improve throughput when integrated into a parallelized architecture. Future work should therefore focus on decoupling model inference from packet processing, enabling streaming execution, and exploring optimized packet manipulation libraries and hardware acceleration strategies. Techniques such as multiprocessing, model quantization, and hardware-aware deployment could further improve end-to-end performance and enable real-time operation under realistic network conditions. Bridging this gap is essential to move from offline evaluation to practical, inline deployment scenarios.

6.2.5 Reinforcement Learning

A further promising direction is the integration of reinforcement learning into the adversarial generation pipeline. By allowing the model to interact with the target NIDS and receive feedback on whether generated packets successfully evade detection, the generator could iteratively refine its perturbation strategies without relying heavily on labeled training data. This would reduce the dependency on large datasets, lower resource costs, and potentially improve evasion rates across a wider range of target systems and network environments. Such an approach could also enable adaptive attacks that evolve in response to changing detection models, further increasing their practical relevance.

Bibliography

[1] D. Han, Z. Wang, Y. Zhong, W. Chen, J. Yang, S. Lu, X. Shi, and X. Yin, "Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors," *IEEE Journal on Selected Areas in Communications*, vol. 39, 2021.

[2] R. Boutaba, M. A. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, and O. M. Caicedo, "A comprehensive survey on machine learning for networking: evolution, applications and research opportunities," *Journal of Internet Services and Applications*, vol. 9, 2018.

[3] I. Goodfellow, P. McDaniel, and N. Papernot, "Making machine learning robust against adversarial inputs: Such inputs distort how machine-learningbased systems are able to function in the world as it is," *Communications of the ACM*, vol. 61, 2018.

[4] B. Biggio and F. Roli, "Wild patterns: Ten years after the rise of adversarial machine learning," *Pattern Recognition*, vol. 84, 2018.

[5] A. E. Cinà, K. Grosse, A. Demontis, S. Vascon, W. Zellinger, B. A. Moser, A. Oprea, B. Biggio, M. Pelillo, and F. Roli, "Wild patterns reloaded: A survey of machine learning security against training data poisoning," *ACM Computing Surveys*, vol. 55, 2023.

[6] M. J. Hashemi, G. Cusack, and E. Keller, "Towards evaluation of nidss in adversarial setting," in *Big-DAMA 2019 - Proceedings of the 3rd ACM CoNEXT Workshop on Big DAta, Machine Learning and Artificial Intelligence for Data Communication Networks, Part of CoNEXT 2019*, 2019.

[7] K. Bock, X. Qiang, G. Hughey, and D. Levin, "Geneva: Evolving censorship evasion strategies," in *Proceedings of the ACM Conference on Computer and Communications Security*, 2019.

[8] F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro, "Intriguing properties of adversarial ml attacks in the problem space," in *Proceedings - IEEE Symposium on Security and Privacy*, vol. 2020-May, 2020.

[9] P. T. Duy, L. K. Tien, N. H. Khoa, D. T. T. Hien, A. G. T. Nguyen, and V. H. Pham, "Digfupas: Deceive ids with gan and function-preserving on adversarial samples in sdn-enabled networks," *Computers and Security*, vol. 109, 2021.

[10] B.-E. Zolbayar, R. Sheatsley, P. McDaniel, M. J. Weisman, S. Zhu, S. Zhu, and S. Krishnamurthy, "Generating practical adversarial network traffic flows using nidsgan," 3 2022. [Online]. Available: https://arxiv.org/abs/2203.06694v1

[11] P. Sun, S. Li, J. Xie, H. Xu, Z. Cheng, and R. Yang, "Gpmt: Generating practical malicious traffic based on adversarial attacks with little prior knowledge," *Computers and Security*, vol. 130, 2023.

[12] H. Liu, A. F. Diallo, and P. Patras, "Amoeba: Circumventing ml-supported network censorship via adversarial reinforcement learning," *Proceedings of the ACM on Networking*, vol. 1, 2023.

[13] P. V. S. Reddy, Y. Vivek, G. Pranay, and V. Ravi, "Chaotic variational auto encoder-based adversarial machine learning," 2023. [Online]. Available: https://arxiv.org/abs/2302.12959

[14] M. Jin and M. Apostolaki, "Pants: Practical adversarial network traffic samples against ml-powered networking classifiers," 9 2024. [Online]. Available: https://arxiv.org/abs/2409.04691v1

[15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Łukasz Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 2017-December, 2017.

[16] Q. J.R, "Induction of decision trees," *Machine Learning*, 1986.

[17] I. Rish, "An empirical study of the naive bayes classifier," *IJCAI 2001 workshop on empirical methods in artificial intelligence*, vol. 22230, 2001.

[18] N. S. Altman, "An introduction to kernel and nearest-neighbor nonparametric regression," *American Statistician*, vol. 46, 1992.

[19] W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *The Bulletin of Mathematical Biophysics*, vol. 5, 1943.

[20] Y. Freund and R. E. Schapire, "Large margin classification using the perceptron algorithm," *Machine Learning*, vol. 37, 1999.

[21] D. R. Cox, "The regression analysis of binary sequences," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 21, no. 1, pp. 238–238, 1959. [Online]. Available: https://rss.onlinelibrary.wiley.com/doi/abs/10.1111/j.2517-6161.1959.tb00334.x

[22] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, 1995.

[23] J. MacQueen, “Some methods for classification and analysis of multivariate observations,” in *Proceedings of the fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, 1967.

[24] L. E. Baum and T. Petrie, “Statistical inference for probabilistic functions of finite state markov chains,” *The Annals of Mathematical Statistics*, vol. 37, 1966.

[25] K. Pearson, “Liii. on lines and planes of closest fit to systems of points in space,” *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 2, 1901.

[26] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Advances in Neural Information Processing Systems*, vol. 3, 2014.

[27] D. P. Kingma and M. Welling, “An introduction to variational autoencoders,” p. 307–392, Nov. 2019. [Online]. Available: http://dx.doi.org/10.1561/2200000056

[28] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, 1992.

[29] Y. Yin, Z. Lin, M. Jin, G. Fanti, and V. Sekar, “Practical gan-based synthetic ip header trace generation using netshare,” in *SIGCOMM 2022 - Proceedings of the ACM SIGCOMM 2022 Conference*, 2022.

[30] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *NAACL HLT 2019 - 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies - Proceedings of the Conference*, vol. 1, 2019.

[31] A. Radford, “Improving language understanding by generative pre-training,” *Homology, Homotopy and Applications*, vol. 9, 2018.

[32] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, 2020.

[33] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, “Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” in *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2020.

[34] A. Dietmüller, S. Ray, R. Jacob, and L. Vanbever, “A new hope for network model generalization,” in *HotNets 2022 - Proceedings of the 2022 21st ACM Workshop on Hot Topics in Networks*, 2022.

[35] J. Zhou, W. Kim, Z. Xu, A. M. Rush, and M. Yu, “Netflowgen: Leveraging generative pre-training for network traffic dynamics,” 2024. [Online]. Available: https://arxiv.org/abs/2412.20635

[36] B. I. Rubinstein, B. Nelson, L. Huang, A. D. Joseph, S. H. Lau, S. Rao, N. Taft, and J. D. Tygar, “Antidote: Understanding and defending against poisoning of anomaly detectors,” in *Proceedings of the ACM SIGCOMM Internet Measurement Conference, IMC*, 2009.

[37] B. Biggio, B. Nelson, and P. Laskov, “Poisoning attacks against support vector machines,” in *Proceedings of the 29th International Conference on Machine Learning, ICML 2012*, vol. 2, 2012.

[38] H. Xiao, B. Biggio, G. Brown, G. Fumera, C. Eckert, and F. Roli, “Is feature selection secure against training data poisoning?” in *32nd International Conference on Machine Learning, ICML 2015*, vol. 2, 2015.

[39] N. Papernot, P. McDaniel, and I. Goodfellow, “Transferability in machine learning: from phenomena to black-box attacks using adversarial samples,” 5 2016. [Online]. Available: http://arxiv.org/abs/1605.07277

[40] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, “Intriguing properties of neural networks,” in *2nd International Conference on Learning Representations, ICLR 2014 - Conference Track Proceedings*, 2014.

[41] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” in *3rd International Conference on Learning Representations, ICLR 2015 - Conference Track Proceedings*, 2015.

[42] A. Globerson and S. Roweis, “Nightmare at test time: Robust learning by feature deletion,” in *ACM International Conference Proceeding Series*, vol. 148, 2006.

[43] O. Dekel, O. Shamir, and L. Xiao, “Learning to classify with missing and corrupted features,” *Machine Learning*, vol. 81, 2010.

[44] G. F. Cretu, A. Stavrou, M. E. Locasto, S. J. Stolfo, and A. D. Keromytis, “Casting out demons: Sanitizing training data for anomaly sensors,” in *Proceedings - IEEE Symposium on Security and Privacy*, 2008.

[45] J. Steinhardt, P. W. Koh, and P. Liang, “Certified defenses for data poisoning attacks,” in *Advances in Neural Information Processing Systems*, vol. 2017-December, 2017.

[46] D. Meng and H. Chen, “Magnet: A two-pronged defense against adversarial examples,” in *Proceedings of the ACM Conference on Computer and Communications Security*, 2017.

[47] B. Biggio, G. Fumera, and F. Roli, “Multiple classifier systems for robust classifier design in adversarial environments,” *International Journal of Machine Learning and Cybernetics*, vol. 1, 2010.

[48] B. Biggio, I. Corona, G. Fumera, G. Giacinto, and F. Roli, “Bagging classifiers for fighting poisoning attacks in adversarial classification tasks,” in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 6713 LNCS, 2011.

[49] E. S. Yu and C. Y. Chen, “Traffic prediction using neural networks,” in *IEEE Global Telecommunications Conference*, vol. 2. Publ by IEEE, 1993, pp. 991–995.

[50] T. T. T. Nguyen, G. Armitage, P. Branch, and S. Zander, “Timely and continuous machine-learning-based classification for interactive ip traffic,” *IEEE/ACM Transactions on Networking*, vol. 20, 2012.

[51] A. Förster and A. L. Murphys, “Froms: Feedback routing for optimizing multiple sinks in wsn with reinforcement learning,” in *Proceedings of the 2007 International Conference on Intelligent Sensors, Sensor Networks and Information Processing, ISSNIP*, 2007.

[52] J. Liu, I. Matta, and M. Crovella, “End-to-end inference of loss nature in a hybrid wired/wireless environment,” 02 2003.

[53] A. Pellegrini, P. D. Sanzo, and D. R. Avresky, “A machine learning-based framework for building application failure prediction models,” in *Proceedings - 2015 IEEE 29th International Parallel and Distributed Processing Symposium Workshops, IPDPSW 2015*, 2015.

[54] M. S. Mushtaq, B. Augustin, and A. Mellouk, “Empirical study based on machine learning approach to assess the qos/qoe correlation,” in *2012 17th European Conference on Network and Optical Communications, NOC 2012, 7th Conference on Optical Cabling and Infrastructure, OC and I 2012*, 2012.

[55] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, “Kitsune: An ensemble of autoencoders for online network intrusion detection,” in *25th Annual Network and Distributed System Security Symposium, NDSS 2018*, 2018.

[56] M. Nasr, A. Bahramali, and A. Houmansadr, “Defeating dnn-based traffic analysis systems in real-time with blind adversarial perturbations,” in *Proceedings of the 30th USENIX Security Symposium*, 2021.

[57] F. Le, M. Srivatsa, R. Ganti, and V. Sekar, “Rethinking data-driven networking with foundation models: Challenges and opportunities,” in *HotNets 2022 - Proceedings of the 2022 21st ACM Workshop on Hot Topics in Networks*, 2022.

[58] A. Feldmann, *Characteristics of TCP Connection Arrivals*. John Wiley Sons, Ltd, 2000, ch. 15, pp. 367–399. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/047120644X.ch15

[59] J. Cao, W. S. Cleveland, D. Lin, and D. X. Sun, "On the nonstationarity of internet traffic," in *Proceedings of the 2001 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, ser. SIGMETRICS '01. New York, NY, USA: Association for Computing Machinery, 2001, p. 102–112. [Online]. Available: https://doi.org/10.1145/378420.378440

[60] "Cicflowmeter (formerly iscxflowmeter)." [Online]. Available: https://www.unb.ca/cic/research/applications.html#CICFlowMeter

[61] "Python version of cicflowmeter." [Online]. Available: https://github.com/hieulw/cicflowmeter

[62] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *ICISSP 2018 - Proceedings of the 4th International Conference on Information Systems Security and Privacy*, vol. 2018-January, 2018.

[63] "Kitsune Network Attack," UCI Machine Learning Repository, 2019, DOI: https://doi.org/10.24432/C5D90Q.